1 一、AutoML与超参数优化
1.1 自动化机器学习AutoML
1.1.1 什么是Auto ML
1.1.2 AutoML的三大成熟研究领域
1.1.3 AutoML的新兴研究领域
1.1.4 AutoML是否会替代算法从业者
2 二、超参数优化HPO方法
2.1 枚举网格搜索(Exhaustive Grid Search)
2.1.1 枚举网格搜索基本原理：
2.1.2 实现枚举网格搜索
2.1.3 枚举网格搜索的理论极限
2.2 随机网格搜索（Random Grid Search)
2.2.1 随机网格搜索基本原理
2.2.2 实现随机网格搜索
2.2.3 随机网格搜索的理论极限
2.2.4 更大的全域空间
2.2.5 连续型的参数空间
2.3 对半网格搜索（Halving Grid Search）
2.3.1 对半网格搜索基本原理
2.3.2 实现对半网格搜索
3 三、bayes优化
3.1 parameter tuning with hyperopt
3.1.1 abstract
3.1.2 Objective Functions - A Motivating Example
3.1.3 More Complicated Examples
3.1.4 Search Spaces
3.1.5 Capturing Info with Trials
3.1.6 Visualization
3.1.7 The Iris Dataset
3.1.8 K-Nearest Neighbors
3.1.9 Support Vector Machines (SVM)
3.1.10 Decision Trees
3.1.11 Random Forests
3.1.12 All Together Now
3.1.13 Conclusion
3.2 lightgbm 分类例子

1 一、AutoML与超参数优化

1.1 自动化机器学习AutoML

1.1.1 什么是Auto ML

- 在过去十年内，机器学习领域获得了巨大的进展，得益于互联网（数据）与芯片（算力）行业的快速发展，各类机器学习算法终于化理论为实践，被应用当各行各业、千家万户。如今，越来越多的科研领域依赖机器学习算法，机器学习算法的实现也因sklearn等算法库而变得轻便简单，但真正高水平的应用却很大程度依赖于机器学习算法工程师的经验。
- **现代机器学习系统不止是关系到编程、数学等技能，更是伴随着众多基于经验和专业性的设计决策**，包括
    1. 如何构筑特征
    2. 选择模型
    3. 如何设置超参数空间（parameter space）
    4. 如何设置神经网络结构
    5. 如何排布和管理数据处理管道（pipeline）

  等等。在实际的机器学习系统搭建过程当中，有超过80%的时间都在处理基于经验的设计决策，而决策所需的专业性拉开了算法工程专家与普通编程者的距离，也拉开了算法专业人员与业务人员的距离。
- 虽然机器学习领域的高专业性保障了算法工程师的稀缺性与高待遇，但在实际商业场景中，还有众多不关注机器学习技术细节、只关注机器学习产出成果的公司与个人，对他们而言，机器学习领域对专业知识的要求成为了巨大的门槛。无论一门技术多么先进，最终的目的还是要围绕业务场景、解决业务问题，而机器学习算法的核心价值也在于能够一定程度上代替人脑完成决策、完成部分思考流程的自动化。为了机器学习能够被更广泛地应用、更快速地发展，研究者们产生了**让机器学习完成机器学习建模流程中所需决策**的想法，正是一想法催生了自动化机器学习AutoML这一研究领域。

  > **AutoML旨在让算法完成机器学习建模流程中所需的决策**
  >
  > **AutoML的最终目标是让机器学习建模流程不再需要人工干预**
  > **或至少不再需要富有经验的机器学习专家的干预**

- 在整个建模流程中，可能自动化的步骤包括了
    1. 数据预处理
    2. 特征构建/特征工程
    3. 模型选择
    4. 搭建神经网络架构
    5. 模型调参
    6. 模型融合

7. 模型可视化

- 其中某些领域在自动化之后，甚至可能完全替代算法工程师的作用。围绕所有可能的领域，学者们艰难探索了数十年。随着人工智能的兴起，AutoML也取得了极大的进展，终于有了较为成熟的方法与工具。现在，AutoML已经是近年来机器学习领域最受关注的研究领域之一：

    1. 学术界

    > - 2021年，google scholar上新增了2690篇autoML领域相关的论文
    > - 光是在自动化神经网络架构领域，就有数百篇论文被发表
    > - 学术研究中几乎不再出现手动调参方法
    > - 自2014年以来，每两年会有一场AutoML或AutoDL大规模挑战赛
    > - 第一届AutoML国际会议将在2022年夏季举办

    从此AutoML不再是混迹在CVPR、ICML等大型会议上的小展台了。

    2. 工业界

    > - 2周之前，华为天才少年钟钊公布了他们的首个AutoML大规模商用的成果
    > - 微软（Azune）、谷歌、亚马逊都已经提出了各自的AutoML解决方案
    > - 2018年，自动化设计的神经网络架构首次胜过人为设计的神经网络架构
    > - 自动化神经网络架构设计已经是许多实验室/高科技公司的标准化技能

    3. 已实现的自动化框架

    > - 自动化数据预处理框架：**MLBoX**
    > - 自动化模型选择框架：**H20 AutoML**
    > - 自动化超参数优化框架：**Bayesian Optimization**，**Hyperopt**，**DEAP**，**Optuna**
    > - 自动化stacking融合：**AutoGluon**
    > - 自动化管道结构优化（基于遗传算法）：**TPOT**
    > - 自动化神经网络架构构建：**Auto-PyTorch**，**AutoKeras**
    > - 综合性自动化建模：**AutoWEKA**，**Auto-sklearn**，**hpsklearn**

- `AutoML正在蓬勃发展，很可能即将迎来腾飞，但在发展过程中，AutoML应用逐渐偏离了“非专业人士”的领`

`域`

例如，业务人员或非专业人员恐怕很难在琳琅满目的第三方框架中选择适合当下项目的框架，同时非专业人员也无法辨别“自动化”究竟完成了哪些工作，还有哪些非自动化的流程需要完成，因此非专业人员只能够使用综合性自动化建模方案。然而：

1. **AutoWEKA只支持Java**
   AutoWEKA底层是基于WEKA所构建，而WEKA只有Java才能调用，因此AutoWEKA拓展性较差，不适用Java以外语言
2. **Auto-sklearn不支持windows**
   Auto-sklearn底层是基于linux中的resorce模块运行，因此不支持Windows系统，也不能完全支持Mac系统
3. **hpsklearn缺乏维护，年久失修**
   hpsklearn底层是基于sklearn，代码简单但缺乏维护，年久失修，时至今日一些基本的代码（如fit）运行还会报错

更糟糕的是，自动化流程是脆弱的，为了能够让用户简单地进行调用，自动化框架往往封装程度很深，一旦框架本身不那么完美，自动化流程就非常容易崩溃。即便真的能够进行调试，无论是最简单的网格搜索，还是复杂的贝叶斯优化，底层都是复杂的数学理论。即便对于机器学习算法工程师来说，AutoML算法背后的原理都令人望而生畏，更不要提完全不理解机器学习的非专业人员了。

1.1.2 AutoML的三大成熟研究领域

1.1.2.1 超参数优化HPO

- `HyperParameter Optimization`
- 参数优化是AutoML领域中最成熟、最深入、也是最知名的方向。每一个机器学习算法都会有超参数，而超参数的设置很大程度上影响了算法实际的使用效果，因此调参是机器学习算法工程师最为基础和重要的任务。

  理论上来说，当算力与数据足够时，HPO的性能一定是超过人类的。HPO能够降低人为工作量，并且HPO得出的结果比认为搜索的复现可能性更高，所以HPO可以极大程度提升科学研究的复现性和公平性。当代超参数优化算法主要可以分为：

  1. 基于网格的各类搜索（Grid）
  2. 基于贝叶斯优化的各类优化算法（Baysian）
  3. 基于梯度的各类优化（Gradient-based）
  4. 基于进化算法的各类优化（Evolutionary Optimzation）

1.1.2.2 神经网络架构搜索NAS

- `Neural Architecture Search`
- 神经网络架构搜索解决深度学习世界当中最关键的问题之一：如何寻找性能良好的深度神经网络架构。与机器学习领域不同，深度学习领域没有一统天下的算法架构，面临任意学习任务，深度学习算法工程师都必须找出适应当前数据和业务的架构。自动化架构搜索可以大大加快新的深度学习应用程序的开发，因为开发人员终于不需要煞费苦心地评估不同的架构了。

1.1.2.3 元学习

- `Meta-Learning`
- 元学习是一种基于其他任务的经验进行学习的方法。人类在学习新知识时，总是会或多或少地借助以前学习过的相关知识。例如，当你学习游泳时，你并不需要从“什么是水”开始，你已经自然地理解了“什么是水”，“水下不能呼吸”、“人下水会沉”等基本信息，然后就可以从“如何漂浮”以及“如何在水下换气”开始学习。

  但对机器学习算法来说，每次学习都是从0开始，在接触训练数据之前，机器学习算法不会有任何“相关经验和理解”。元学习就是赋予机器学习算法“相关经验”的方法。在元学习中，我们一般会先让算法学习与目标任务相关的其他任务或其他信息，以获得能够知道检索新任务最优模型的相关知识，然后再在目标任务上进行训练。这一手段被广泛地应用于HPO和NAS领域，同时深度学习中的“迁移学习”也于元学习有异曲同工之妙。

  在实际应用中，原学习可以被用于以下任务：

    1. HPO&NAS
    2. 算法预测
    3. 模型选择
    4. 算法构建（L2L）

1.1.3 AutoML的新兴研究领域

1. **交互式可解释性机器学习ixAutoML**（Interactive and Explainable AutoML）

   大部分AutoML框架封装程度很深、许多AutoML算法还是黑箱算法，因此无论是机器学习工作者、还是非专业人士，都不太信任AutoML的结果。ixAutoML是研究自动化解释模型、自动化可视化的领域，旨在增强专业人员对AutoML的信任感，增强AutoML的透明程度。该领域已经发展了一段时间，目前为止最先进的研究能够可视化参数变化对模型整体的影响。

2. **绿色AI**（Green AI）

   AutoML最关键的问题之一是巨大的计算量。当人工神经网络、boosting算法等超参数众多的算法被应用当工业当中，巨量的数据、巨量的超参数让AutoML对算力的要求很高。绿色AI旨在自动研发低参数量、低超参量、但性能优越的算法。这个领域目前刚刚萌芽。

3. **非线性流程优化**

   无论我们如何“自动化”机器学习，在调参和模型选择中都会有一个巨大的争议：有的模型在没有调参之前结果不佳，但是调参之后可能得到很好的结果，有的模型在调参之前结果很好，但是调参之后未必能得到很好的结果。但在自动化过程中，我们一定是先选择模型，再调节参数，因此具有强大调参潜力的模型反而容易在自动化过程中被筛选掉。现在的技术已经支持对多个模型同时调参，并最终选择调参后结果最佳的模型，但这样又会需要大量计算资源与计算时间。如何快速解决模型选择与调参顺序所导致的问题，是现代autoML关键的课题。

1.1.4 AutoML是否会替代算法从业者

- **短时间内一定不会**。在爆发新一轮技术突破之前，AutoML还不能替代算法工程师。相对的，AutoML是辅助算法工程师的关键工具。有以下几个原因：
    1. 适用于业务人员的AutoML框架还不成熟，并且全流程框架方面的研究也未见起色。
    2. 在较大的、复杂的模型（如深度学习，boosting）或者较大的数据集上，AutoML求解虽然准确，但运算代价机器巨大。为节约成本，autoML过程必须经过算法工程师的优化和干涉。在NAS领域，神经网络架构一般也不会从0开始搜索，如何选择正确的备选项，依然依赖于算法工程师的专业知识。
    3. 数据/业务问题的复杂程度目前依然超出AutoML的处理范围，AutoML只能够适用于较为固定的场景例如，损失函数不平滑、或损失函数不是凸函数时，AutoML的优化算法会失效。

---

2 二、超参数优化HPO方法

2.1 枚举网格搜索(Exhaustive Grid Search)

2.1.1 枚举网格搜索基本原理：

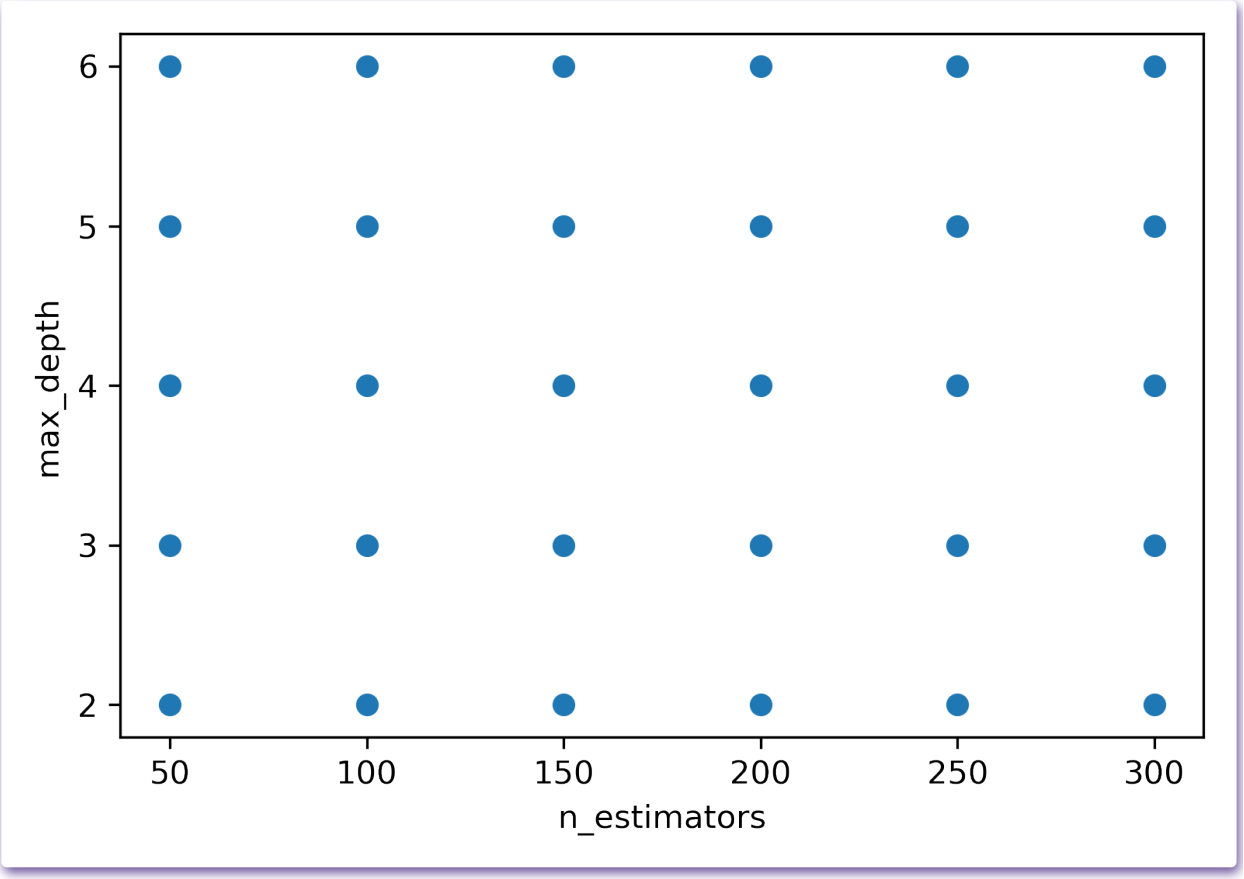
- 基本原理介绍

  在所有超参数优化的算法当中，枚举网格搜索是最为基础和经典的方法。在搜索开始之前，我们需要人工将每个超参数的备选值一一列出，多个不同超参数的不同取值之间排列组合，最终将组成一个参数空间（parameter space）。枚举网格搜索算法会将这个参数空间当中**所有的参数组合**带入模型进行训练，最终选出泛化能力最强的组合作为模型的最终超参数。

  当衡量模型的“泛化能力”时，我们往往会使用交叉验证输出指标的均值和方差，因此超参数优化算法总是与网格搜索密不可分。

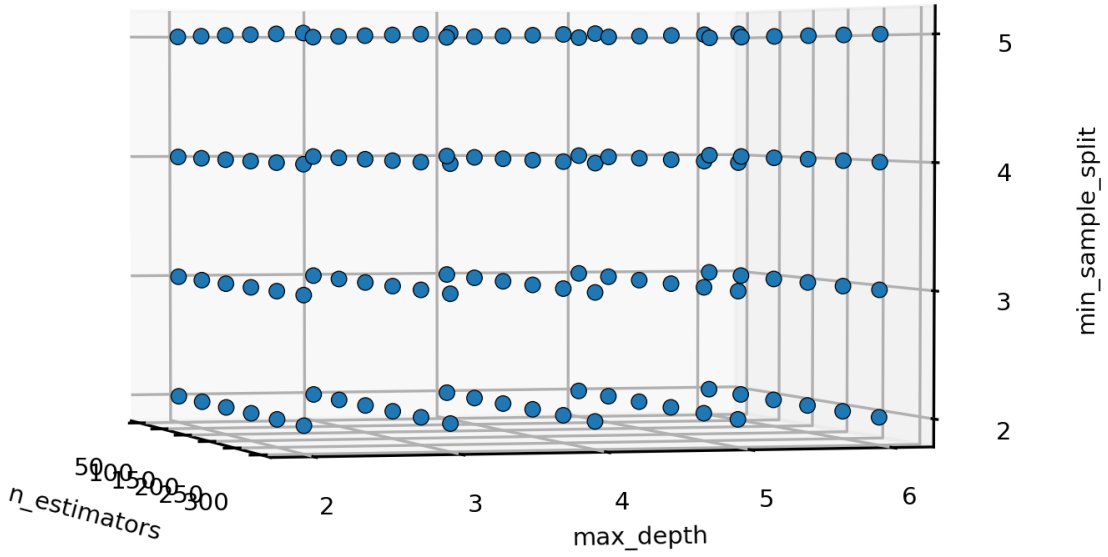
  在进行网格搜索之前，我们需要先确认进行搜索的参数空间。所谓**参数空间**，就是备选参数所构成的空间。以随机森林算法为例，假设我们要调整n_estimators与max_depth，且n_estimators的备选范围是[50,100,150,200,250,300]，max_depth的备选范围是[2,3,4,5,6]，则两个参数可以构成如下二维空间。

```
import matplotlib
import matplotlib.pyplot as plt

n_e_list = [*range(50,350,50)] # n_estimators列表
m_d_list = [*range(2,7)] # max_depth列表
comb = pd.DataFrame([(n_estimators, max_depth) for n_estimators in n_e_list for max_depth in m_d_list])
plt.figure(dpi=300)
plt.scatter(comb.iloc[:,0],comb.iloc[:,1],cmap="Blues")
plt.xticks([*range(50,350,50)])
plt.yticks([*range(2,7)])
plt.xlabel("n_estimators")
plt.ylabel("max_depth")
```



- 假设我们要调整 `n_estimators`， `max_depth` 与 `min_sample_split`， 且min_sample_split的备选参数为[2,3,4,5]，则三个参数可以构成如下三维空间：

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d

m_s_s_list = [2,3,4,5]
X,Y,Z = np.meshgrid(n_e_list,m_d_list,m_s_s_list)
plt.figure(dpi=300)
ax = plt.axes(projection="3d")
ax.view_init(2, -15)
#ax.view_init(90, -30)
ax.scatter(X,Y,Z,cmap="Blues",edgecolor="k",linewidth=0.3,s=20,alpha=1);
ax.set_xticks([*range(50,350,50)])
ax.set_yticks([*range(2,7)])
ax.set_zticks([2,3,4,5])
plt.xlabel("n_estimators",fontsize=7)
plt.ylabel("max_depth",fontsize=7)
ax.set_zlabel("min_sample_split",fontsize=7)
ax.zaxis.set_tick_params(labelsize=7)
ax.xaxis.set_tick_params(labelsize=7)
ax.yaxis.set_tick_params(labelsize=7);
```



在参数空间中，每一个点就是一组参数的组合，枚举网格搜索的目标就是从空间中挑选出最好的参数组合。因此，枚举网格搜索会对空间中每一个点进行建模，以找出令模型整体损失函数最小、泛化能力最强的组合。

> 广义的来说，参数空间中的超参数也可以是评估器，也可以是某个预处理流程中的参数。因此只要编码得当，网格搜索可以被用于模型选择，甚至是自动化预处理。

2.1.2 实现枚举网格搜索

- 在sklearn当中，我们可以很容易地实现网格搜索，以随机森林回归器为例：

```
model_selection.GridSearchCV(estimator # 调参对象
                             , param_grid
                             , scoring=None
                             , n_jobs=None
                             , refit=True
                             , cv=None
                             , verbose=0
                             , pre_dispatch='2n_jobs'
                             , error_score=nan
                             , return_train_score=False
                            )
```

全部参数如下所示，其中加粗的是需要高度关注的参数：

Name	Description
estimator	**调参对象，某评估器**
param_grid	**参数空间，可以是字典或者字典构成的列表，稍后介绍参数空间的创建方法**
scoring	**评估指标，支持同时输出多个参数**
n_jobs	设置工作时参与计算的线程数
refit	挑选评估指标和最佳参数，在完整数据集上进行训练
cv	交叉验证的折数
verbose	输出工作日志形式
pre_dispatch	多任务并行时任务划分数量
error_score	当网格搜索报错时返回结果，选择'raise'时将直接报错并中断训练过程，其他情况会显示警告信息后继续完成训练
return_train_score	在交叉验证中是否显示训练集中参数得分

- 导入数据

```
import time #计时模块time
from sklearn.ensemble import RandomForestRegressor as RFR
from sklearn.model_selection import KFold, GridSearchCV

def RMSE(cvresult,key):
    return (abs(cvresult[key])**0.5).mean()


data = pd.read_csv("./train_encode.csv",index_col=0)

X = data.iloc[:,:-1]
y = data.iloc[:,-1]



X.columns.tolist()

y.describe() #MSE, RMSE

X.head()
```

- 创建参数空间

```
param_grid_simple = {'n_estimators': [*range(5,100,5)]
                     , 'max_depth': [*range(25,36,2)]
                     , "max_features": ["log2","sqrt","auto"]
                    }
```

- 实例化用于搜索的评估器、交叉验证评估器与网格搜索评估器

```
reg = RFR(random_state=100
          ,verbose=True
          ,n_jobs=12
         )

cv = KFold(n_splits=5 # 交叉验证折数
           ,shuffle=True # Whether to shuffle the data before splitting into batches.
           ,random_state=100
          )

search = GridSearchCV(estimator=reg
                      ,param_grid=param_grid_simple
                      ,scoring = "neg_mean_squared_error" #MSE
                      ,verbose = True
                      ,cv = cv
                      ,n_jobs=12
                      )
```

- 训练网格搜索评估器

```
#=====【TIME WARNING: 5~10min】=====#
start = time.time()
search.fit(X,y)
print(time.time() - start)

"""输出结果"""
Fitting 5176.2584252357483 folds for each of 342 candidates, totalling 1710 fits
176.2584252357483
```

- 查看结果

```
search.best_estimator_

"""输出结果"""
RandomForestRegressor(max_depth=27
                      , n_estimators=95
                      , n_jobs=12
                      , random_state=100
                      , verbose=True
                     )
```

```
abs(search.best_score_)**0.5 #RMSE

"""输出结果"""
30308.601328615994
```

2.1.3 枚举网格搜索的理论极限

- 如果参数空间中的某一个点指向了损失函数真正的最小值，那枚举网格搜索时一定能够捕捉到该最小值以及对应的参数（相对的，假如参数空间中没有任意一点指向损失函数真正的最小值，那网格搜索就一定无法找到最小值对应的参数组合）

  而参数空间越大、越密，参数空间中的组合刚好覆盖损失函数最小值点的可能性就会越大。**这是说，极端情况下，当参数空间穷尽了所有可能的取值时，网格搜索一定能够找到损失函数的最小值所对应的最优参数组合，且该参数组合的泛化能力一定是强于人工调参的**。

- `枚举网格搜索的限制：算力与时间`

  是，参数空间越大，网格搜索所需的算力和时间也会越大，当参数维度上升时，网格搜索所需的计算量更是程指数级上升的：

  1. 只有1个参数n_estimators，备选范围是[50,100,150,200,250,300]，需要建模**6**次。
  2. 增加参数max_depth，且备选范围是[2,3,4,5,6]，需要建模**30**次。
  3. 增加参数min_sample_split，且备选范围为[2,3,4,5]，需要建模**120**次。

  同时，参数优化的目标是找出令模型泛化能力最强的组合，因此需要交叉验证来体现模型的泛化能力，假设交叉验证次数为5，则三个参数就需要建模600次。在面对超参数众多、且超参数取值可能无限的人工神经网络、融合模型、集成模型时，伴随着数据和模型的复杂度提升，网格搜索所需要的时间会急剧增加，完成一次枚举网格搜索可能需要耗费几天几夜。因此当我们使用网格搜索时，要尽量精简参数空间，以节约计算资源。

2.2 随机网格搜索（Random Grid Search)

2.2.1 随机网格搜索基本原理

- 基本原理介绍

  在讲解网格搜索时我们提到，伴随着数据和模型的复杂度提升，网格搜索所需要的时间急剧增加。对于刚才尝试的随机森林算法，如果使用过w的数据，搜索时间则会立刻上升好几个小时。考虑到后续实践过程中，算法和数据都将更加复杂，而建模过程中超参数调优是模型训练的必备环节，因此，我们急需寻找到一种更加高效的超参数搜索方法。

  首先，当所使用的算法确定时，决定枚举网格搜索运算速度的因子一共有两个：

  1. 参数空间的大小：参数空间越大，需要建模的次数越多
  2. 数据量的大小：数据量越大，每次建模时需要的算力和时间越多

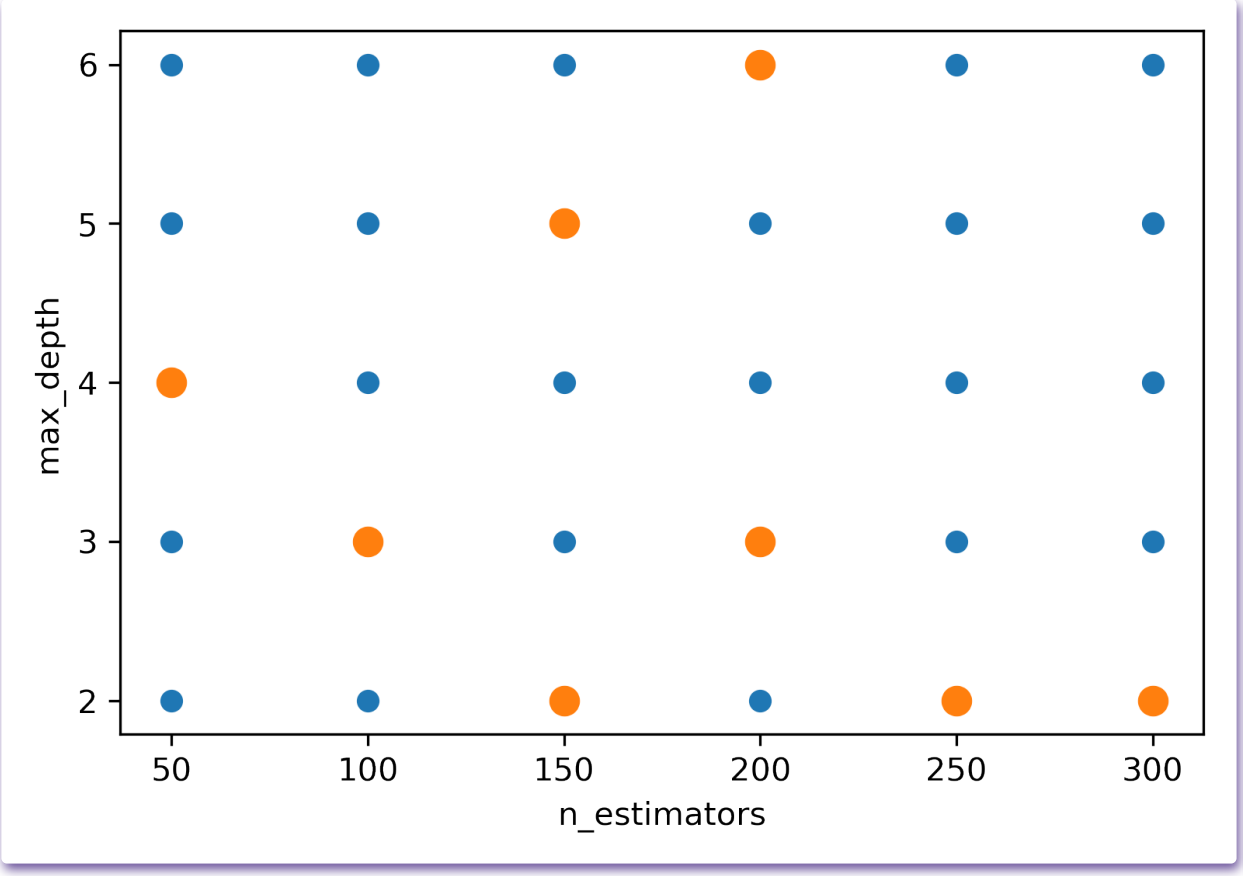
  因此，sklearn中的网格搜索优化方法主要包括两类

  1. `其一是调整搜索空间`
  2. `其二是调整每次训练的数据`

  其中，调整参数空间的具体方法，是放弃原本的搜索中必须使用的全域超参数空间，改为挑选出部分参数组合，构造超参数子空间，并只在子空间中进行搜索。以下图的二维空间为例，枚举网格搜索必须对30种参数组合都进行搜索，而调整搜索空间之后，我们只抽样

出橙色的参数组合作为“子空间”，并只对橙色参数组合进行搜索。如此一来，整体搜索所需的计算量就大大下降了，原本需要30次建模，现在只需要8次建模。

```
n_e_list = [*range(50,350,50)]
m_d_list = [*range(2,7)]
comb = pd.DataFrame([(n_estimators, max_depth) for n_estimators in n_e_list for max_depth in m_d_list])
plt.figure(dpi=300)
plt.scatter(comb.iloc[:,0],comb.iloc[:,1],cmap="Blues")
plt.scatter([50,250,200,200,300,100,150,150],[4,2,6,3,2,3,2,5],cmap="red",s=20,linewidths=5)
plt.xticks([*range(50,350,50)])
plt.yticks([*range(2,7)])
plt.xlabel("n_estimators")
plt.ylabel("max_depth");
```



在sklearn中，随机抽取参数子空间并在子空间中进行搜索的方法叫做随机网格搜索RandomizedSearchCV。由于搜索空间的缩小，需要枚举和对比的参数组的数量也对应减少，整体搜索耗时也将随之减少，因此随机网格搜索的速度比枚举网格搜索快很多。

不过，需要注意的是，**随机网格搜索在实际运行时，并不是先抽样出子空间，再对子空间进行搜索**，而是仿佛“循环迭代”一般，在一次次迭代随机抽取1组参数进行建模，下一次迭代再随机抽取1组参数进行建模，这种随机抽样是不放回的。我们可以控制随机网格搜索的迭代次数，来控制整体被抽出的参数子空间的大小。

2.2.2 实现随机网格搜索

- `sklearn` 中随机搜索的类是 `RandomizedSearchCV`

```
sklearn.model_selection.RandomizedSearchCV(estimator
                                           , param_distributions
                                           , n_iter=10
                                           , scoring=None
                                           , n_jobs=None
                                           , refit=True
                                           , cv=None
                                           , verbose=0
                                           , pre_dispatch='2*n_jobs'
                                           , random_state=None
                                           , error_score=nan
                                           , return_train_score=False
                                          )
```

全部参数解读如下，其中加粗的是随机网格搜索独有的参数：

Name	Description
estimator	调参对象，某评估器
param_distributions	全域参数空间，可以是字典或者字典构成的列表
n_iter	迭代次数，迭代次数越多，抽取的子参数空间越大
scoring	评估指标，支持同时输出多个参数
n_jobs	设置工作时参与计算的线程数
refit	挑选评估指标和最佳参数，在完整数据集上进行训练
cv	交叉验证的折数
verbose	输出工作日志形式
pre_dispatch	多任务并行时任务划分数量
random_state	随机数种子
error_score	当网格搜索报错时返回结果，选择'raise'时将直接报错并中断训练过程，其他情况会显示警告信息后继续完成训练
return_train_score	在交叉验证中是否显示训练集中参数得分

- 我们依然借用之前已经建立好的数据X与y，以及随机森林回归器，来实现随机网格搜索：

```python
import time #计时模块time
from sklearn.ensemble import RandomForestRegressor as RFR
from sklearn.model_selection import KFold, GridSearchCV
from sklearn.model_selection import RandomizedSearchCV

def RMSE(cvresult,key):
    return (abs(cvresult[key])**0.5).mean()


data = pd.read_csv("./train_encode.csv",index_col=0)

X = data.iloc[:,:-1]
y = data.iloc[:,-1]
```

- 创造参数空间 - 使用与网格搜索时完全一致的空间

```python
param_grid_simple = {'n_estimators': [*range(5,100,5)]
                     , 'max_depth': [*range(25,36,2)]
                     , "max_features": ["log2","sqrt","auto"]
                    }
```

- 建立回归器、交叉验证

```python
reg = RFR(random_state=100
          ,verbose=True
          ,n_jobs=12
         )

cv = KFold(n_splits=5
           ,shuffle=True
           ,random_state=100
          )
```

- 计算全域参数空间大小

```python
len([*range(5,100,5)]) * len([*range(25,36,2)]) * 3

"""输出如下"""
342
```

- 定义随机搜索

```python
search = RandomizedSearchCV(estimator=reg
                            ,param_distributions=param_grid_simple
                            ,n_iter = 180 #子空间的大小是全域空间的一半
                            ,scoring = "neg_mean_squared_error"
                            ,verbose = True
                            ,cv = cv
                            ,n_jobs=12)
```

- 训练随机搜索评估器

```
#=====【TIME WARNING: 5~10min】=====#
start = time.time()
search.fit(X,y)
print(time.time() - start)

"""输出如下"""
Fitting 5 folds for each of 180 candidates, totalling 900 fits
89.24192094802856
```

- 查看模型结果

```
search.best_estimator_

"""输出如下"""
RandomForestRegressor(max_depth=27
                      , n_estimators=95
                      , n_jobs=12
                      , random_state=100
                      , verbose=True)
```

```
abs(search.best_score_)**0.5

"""输出如下"""
30308.601328615994
```

HPO方法	网格搜索	随机搜索
全域空间/实际空间	342/342	180/342
搜索时间	176.258	**89.242(↓)**
最优分数（RMSE）	30308.601328615994	30308.601328615994

很明显，在相同参数空间、相同模型的情况下，随机网格搜索的运算速度(89秒)是普通网格搜索（176秒）的一半，当然，这与子空间是全域空间的一半有很大的联系。由于随机搜索只是降低搜索的次数，并非影响搜索过程本身，因此其运行时间基本就等于n_iter/全域空间组合数 * 网格搜索的运行时间。

2.2.3 随机网格搜索的理论极限

- 虽然通过缩小子空间可以提升搜索的速度，但是随机网格搜索的精度看起来并没有削减太多，随机网格搜索可以得到和网格搜索一样好的结果吗？它也像网格搜索一样，可以得到最优的参数组合吗？为什么缩小参数空间之后，随机网格搜索的结果还与网格搜索一致？

  从直觉上来说，我们很难回答这些问题，但我们可以从数学的随机过程的角度来理解这个问题。在机器学习算法当中，有非常多通过随机来降低运算速度（比如Kmeans，随机挑选样本构建簇心）、或通过随机来提升模型效果的操作（比如随机森林，比如小批量随机梯度下降）。随机网格搜索属于前者，而它能够有效的根本原因在于：

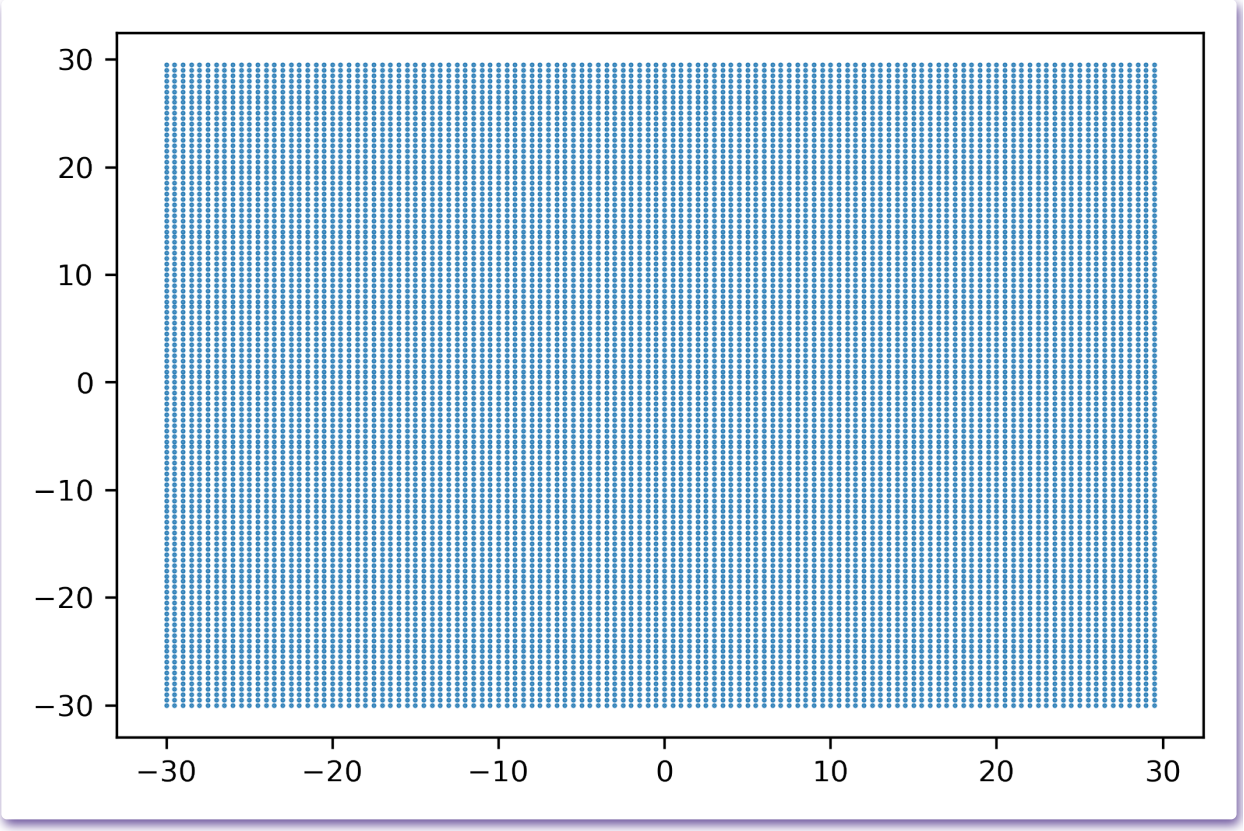
  1. **抽样出的子空间反馈出与全域空间相似的分布**

2. **子空间越大，子空间与全域空间越相似**
3. **在与全域空间相似的空间中，随机搜索即便不能找到真正的最小值，也能找到与最小值高度接近的某一个次小值**

如果全域空间包括了理论上的损失函数最小值，那一个与全域空间分布高度相似的子空间很可能也包括损失函数的最小值，或包括非常接近最小值的一系列值。

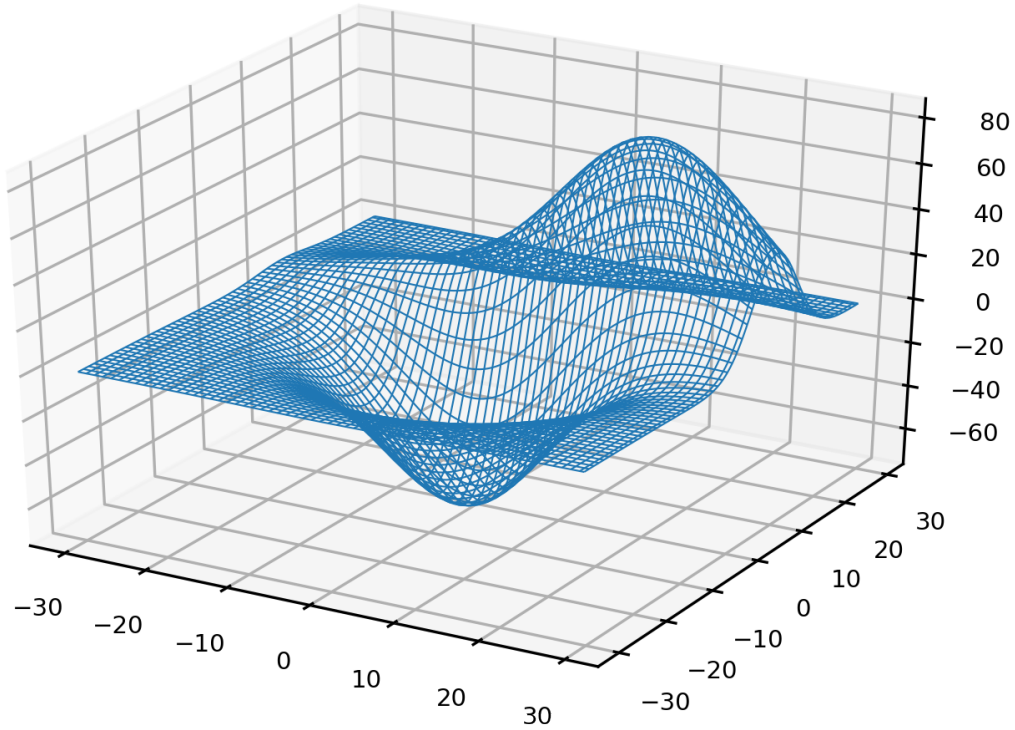
我们可以绘图查看

```
from mpl_toolkits.mplot3d import axes3d

p1, p2, MSE = axes3d.get_test_data(0.05)
#假设现在有两个参数，p1与p2，两个参数组成的参数组合对应着损失函数值MSE
#参数0.05是指参数空间中，点与点之间的距离
#因此该数字越小，取出来的样本越多

len(p1) #参数1有120个取值

len(p2) #Y也有120个取值

MSE.shape #排列组合，则有120*120 = 14400个取值结果

#参数空间
plt.figure(dpi=300)
plt.scatter(p1,p2,s=0.2)
plt.xticks(fontsize=9)
plt.yticks(fontsize=9)
```



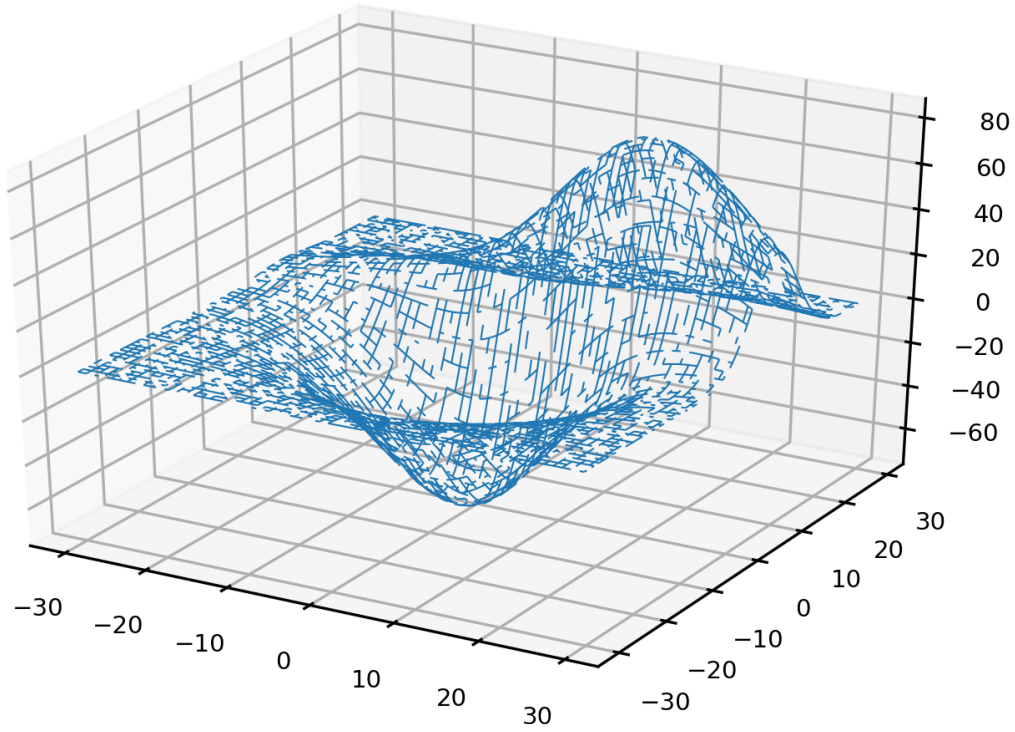

```
#参数与损失共同构建的函数
plt.figure(dpi=300)
```

```
ax = plt.axes(projection="3d")
ax.plot_wireframe(p1
                  ,p2
                  ,MSE
                  ,rstride=2
                  ,cstride=2
                  ,linewidth=0.5
                 )
ax.zaxis.set_tick_params(labelsize=7)
ax.xaxis.set_tick_params(labelsize=7)
ax.yaxis.set_tick_params(labelsize=7);

np.min(MSE) #整个参数中，可获得的MSE最小值是-73.39
```




```
#现在，我们从该空间上抽取子空间
import numpy as np

#我们从空间中抽取n个组合，n越大子空间越大
#现在总共有14400个组合
#对被抽中的点来说，损失函数的值就是MSE，对没有抽中的点来说，损失函数值是空值
#因此，我们只需要找出没有抽中的点，并让它的损失函数值MSE为空就可以了
n = 10000

#从0~14400中生成(14400-n)个随机数，形成没有被抽到子空间中的点的索引
unsampled = np.random.randint(0,14400,14400-n)

p1, p2, MSE = axes3d.get_test_data(0.05)
MSE = MSE.ravel()
MSE[unsampled] = np.nan #没有被抽出的样本，我们让损失函数值为空
```

```
MSE = MSE.reshape((120,120))

#参数与损失共同构建的函数
plt.figure(dpi=300)
ax = plt.axes(projection="3d")
ax.plot_wireframe(p1,p2,MSE,rstride=2,cstride=2,linewidth=0.5)
ax.zaxis.set_tick_params(labelsize=7)
ax.xaxis.set_tick_params(labelsize=7)
ax.yaxis.set_tick_params(labelsize=7);

#求出当前损失函数上的最小值
MSE = MSE.ravel().tolist()
MSE = [x for x in MSE if str(x) != 'nan']
print(np.min(MSE)) # -73.39620971601681
```



很容易发现，当抽取的子空间越大时，子空间的分布会与全域空间越来越相似。同时，即便抽取的子空间非常小，子空间中的分布还是很大程度上与全域空间相似，并且函数的最小值、或与最小值高度接近的某个值，一定会被包括在子空间当中。

当然了，这是基于全域空间非常大的情况，如果全域空间本来就比较小（点分布比较稀疏），在抽样时抽出损失函数最小值的可能性就会比较小，随机搜索结果相应的也会较差。

我们可以在图像上验证如下事实：

1. 抽样出的子空间可以一定程度上反馈出全域空间的分布
2. 子空间相对越大（含有的参数组合数越多），子空间的分布越接近全域空间的分布
3. 当全域空间本身足够密集时，很小的子空间也能获得与全域空间相似的分布

4. 如果全域空间包括了理论上的损失函数最小值，那一个与全域空间分布高度相似的子空间很可能也包括损失函数的最小值，或包括非常接近最小值的一系列次小值
5. 在与全域空间分布相似的子空间中，随机搜索即便不能找到真正的最小值，也有很高的概率找到与最小值高度接近的某一个次小值

因此，只要子空间足够大，随机网格搜索的效果一定是高度逼近枚举网格搜索的。**在全域参数空间固定时，随机网格搜索可以在效率与精度之间做权衡**。子空间越大，精度越高，子空间越小，效率越高。

2.2.4 更大的全域空间

- 不过，由于随机网格搜索计算更快，所以在相同计算资源的前提下，我们可以对随机网格搜索使用更大的全域空间，因此随机搜索可能得到比网格搜索更好的效果：
- 导入数据

```
import time #计时模块time
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor as RFR
from sklearn.model_selection import KFold, GridSearchCV
from sklearn.model_selection import RandomizedSearchCV

def RMSE(cvresult,key):
    return (abs(cvresult[key])**0.5).mean()


data = pd.read_csv("./train_encode.csv",index_col=0)

X = data.iloc[:,:-1]
y = data.iloc[:,-1]
```

- 创造参数空间 - 比网格搜索大一倍的全域空间

```
param_grid_simple = {'n_estimators': [*range(5,100,5)]
                     , 'max_depth': [*range(25,36,2)]
                     , "max_features": ["log2","sqrt",16,32,64,"auto"]
                    }
#计算全域参数空间大小，这是我们能够抽样的最大值
len([*range(5,100,5)]) * len([*range(25,36,2)]) * 6


"""输出如下"""
684
```

- 建立回归，交叉验证，定义随机搜索

```
#建立回归器、交叉验证
reg = RFR(random_state=100,verbose=True,n_jobs=12)
cv = KFold(n_splits=5,shuffle=True,random_state=1412)

#定义随机搜索
search = RandomizedSearchCV(estimator=reg
                            ,param_distributions=param_grid_simple
                            ,n_iter = 360 #使用与枚举网格搜索类似的拟合次数
                            ,scoring = "neg_mean_squared_error"
                            ,verbose = True
                            ,cv = cv
                            ,n_jobs=12)
```

- 训练随机搜索评估器

```
#===== 【TIME WARNING: 5~10min】 =====#
start = time.time()
search.fit(X,y)
print(time.time() - start)

"""输出如下"""
Fitting 5 folds for each of 360 candidates, totalling 1800 fits
201.28360509872437
```

- 查看结果

```
#查看最佳评估器
search.best_estimator_

"""输出如下"""
RandomForestRegressor(max_depth=29
                      , max_features=16
                      , n_estimators=95
                      , n_jobs=12
                      , random_state=100
                      , verbose=True)
```

```
#查看最终评估指标
abs(search.best_score_)**0.5

"""输出如下"""
29753.904000901704
```

HPO方法	网格搜索	随机搜索	随机搜索
全域空间/实际空间	342/342	180/342	360/684
搜索时间	176.258	**89.242(↓)**	201.284
最优分数（RMSE）	30308.601328615994	30308.601328615994	29753.904000901704

可以发现，当全域参数空间增大之后，随即网格搜索可以使用与小空间上的网格搜索相似的时间，来求得更好的结果。除了可以容忍更大的参数空间之外，随机网格搜索还可以接受连续性变量作为参数空间的输入。

2.2.5 连续型的参数空间

- 对于网格搜索来说，参数空间中的点是分布均匀、间隔一致的，因为网格搜索无法从某种“分布”中提取数据，只能使用组合好的参数组合点，而随机搜索却可以接受“分布”作为输入。如上图所示，对于网格搜索来说，损失函数的最低点很不幸的、位于两组参数之间，在这种情况下，枚举网格搜索是100%不可能找到最小值的。但对于随机网格搜索来说，由于是一段分布上随机选择参数点，因此在同样的参数空间中，取到更好的值的可能性更大。
- uniform是均匀分布，默认生成[0,1]之间的数字，可以使用loc来调整起点，scale来调整终点。我们还可以选择其他的分布，比如指数分布expon, gamma分布, 或者是randint。注意scipy这里并不是像np.linspace()一样生成一段离散的数字，而是直接生成一个分布对象。并且，**我们并没有在分布对象中指定尺寸，也就是说，在这段分布上究竟要取几个随机的参数备选值，是由随机搜索自己决定的**。理论上来说，我们给出的n_iter越大，任意参数的分布上可能被取到的点就越多。

  在之前调整随机森林的时候，我们给出的所有参数都是只能接纳正整数的参数，因此在这里我们可以使用scipy.stats.randint，不过randint并不是严格意义上的连续分布。

  严格来说，连续型搜索更适用于学习率，C，alpha这样的参数（无上限，以浮点数为主），随机森林的参数中最接近这个定义的是min_impurity_decrease，表示决策树在分枝是可以容忍的最小的不纯度下降量。我们借着这个参数，使用均匀分布来进行搜索试试看。

```
import scipy #使用scipy来帮助我们建立分布


param_grid_simple = {'n_estimators': [*range(5,100,5)]
                     , 'max_depth': [*range(25,36,2)]
                     , "max_features": ["log2","sqrt",16,32,64,"auto"]
                     , "min_impurity_decrease" : scipy.stats.uniform(1,200)
                     }
#建立回归器、交叉验证
reg = RFR(random_state=100,verbose=True,n_jobs=12)
cv = KFold(n_splits=5,shuffle=True,random_state=100)

#定义随机搜索
search = RandomizedSearchCV(estimator=reg
                            ,param_distributions=param_grid_simple
                            ,n_iter = 360 #还是使用360这个搜索次数
                            ,scoring = "neg_mean_squared_error"
                            ,verbose = True
                            ,cv = cv
                            ,n_jobs=12)
```

```python

#训练随机搜索评估器
#=====【TIME WARNING: 5~10min】=====#
start = time.time()
search.fit(X,y)
print(time.time() - start)

"""输出如下"""
Fitting 5 folds for each of 360 candidates, totalling 1800 fits
183.32190084457397
```

- 查看最佳评估器

```python
search.best_estimator_

"""输出如下"""
RandomForestRegressor(max_depth=25
                      , max_features=16
                      , min_impurity_decrease=198.83322638931952
                      , n_estimators=40
                      , n_jobs=12
                      , random_state=100
                      , verbose=True
                     )
```

```python
#查看最终评估指标
abs(search.best_score_)**0.5

"""输出如下"""
29136.250762344902
```

HPO方法	网格搜索	随机搜索	随机搜索	随机搜索（连续型）
全域空间/实际空间	342/342	180/342	360/684	360/无限
搜索时间	176.258	**89.242(↓)**	201.284	183.322
最优分数（RMSE）	30308.601328615994	30308.601328615994	29753.904000901704	29136.250762344902

理论上来说，当枚举网格搜索所使用的全域参数空间足够大/足够密集时，枚举网格搜索的最优解是随机网格搜索的上限，因此 **理论上随机网格搜索不会得到比枚举网格搜索更好的结果** 。

但现实中的问题是，由于枚举网格搜索的速度太慢，因此枚举网格搜索的全域参数空间往往无法设置得很大，也无法设置得很密集，因此网格搜索的结果很难接近理论上的最优值。当随机网格搜索将空间设置更大、更密集时，就可以捕获更广空间的分布，也自然就可能捕获到理论上的最优值了。

2.3 对半网格搜索（Halving Grid Search）

2.3.1 对半网格搜索基本原理

- 在讲解随机网格搜索之前，我们梳理了决定枚举网格搜索运算速度的因子：
    1. 参数空间的大小：参数空间越大，需要建模的次数越多
    2. 数据量的大小：数据量越大，每次建模时需要的算力和时间越多

  面对枚举网格搜索过慢的问题，sklearn中呈现了两种优化方式：
    1. 其一是**调整搜索空间**
    2. 其二是调整**每次训练的数据**。

  调整搜索空间的方法就是随机网格搜索，而调整每次训练数据的方法就是对半网格搜索。

  假设现在存在数据集D，我们从数据集D中随机抽样出一个子集d。如果一组参数在整个数据集DD上表现较差，那大概率这组参数在数据集的子集d上表现也不会太好。反之，如果一组参数在子集d上表现不好，我们也不会信任这组参数在全数据集D上的表现。

  > 参数在子集与全数据集上反馈出的表现一致

  如果这一假设成立，那在网格搜索中，比起每次都使用全部数据来验证一组参数，或许我们可以考虑只带入训练数据的子集来对超参数进行筛选，这样可以极大程度地加速我们的运算。

- 但在现实数据中，这一假设要成立是有条件的，即**任意子集的分布都与全数据集D的分布类似。当子集的分布越接近全数据集的分布，同一组参数在子集与全数据集上的表现越有可能一致**。根据之前在随机网格搜索中得出的结论，我们知道子集越大、其分布越接近全数据集的分布，但是大子集又会导致更长的训练时间，因此为了整体训练效率，我们不可能无限地增大子集。这就出现了一个矛盾：大子集上的结果更可靠，但大子集计算更缓慢。对半网格搜索算法设计了一个精妙的流程，可以很好的权衡子集的大小与计算效率问题，我们来看具体的流程：
    1. 首先从全数据集中无放回随机抽样出一个很小的子集d_0，并在d_0上验证全部参数组合的性能。根据d_0上的验证结果，淘汰评分排在后1/2的那一半参数组合
    2. 然后，从全数据集中再无放回抽样出一个比d_0大一倍的子集d1d1，并在d_1上验证剩下的那一半参数组合的性能。根据d_1上的验证结果，淘汰评分排在后1/2的参数组合
    3. 再从全数据集中无放回抽样出一个比d_1大一倍的子集d_2，并在d_2上验证剩下1/4的参数组合的性能。根据d2d2上的验证结果，淘汰评分排在后1/2的参数组合……

  持续循环。如果使用S代表首次迭代时子集的样本量，C代表全部参数组合数，则在迭代过程中，用于验证参数的数据子集是越来越大的，而需要被验证的参数组合数量是越来越少的:

迭代次数	子集样本量	参数组合数
1	S	C
2	2S	$\frac{1}{2}$ C
3	4S	$\frac{1}{4}$ C
4	8S	$\frac{1}{8}$ C
……		

（当C无法被除尽时，则向上取整）

当备选参数组合只剩下一组，或剩余可用的数据不足，循环就会停下，具体地来说，当 $\frac{1}{n}$ C <= 1或者nS > 总体样本量，搜索就会停止。在实际应用时，哪一种停止条件会先被触发，需要看实际样本量及参数空间地大小。同时，每次迭代时增加的样本量、以及每次迭代时不断减少的参数组合都是可以自由设定的。

在这种模式下，只有在不同的子集上不断获得优秀结果的参数组合能够被留存到迭代的后期，最终选择出的参数组合一定是在所有子集上都表现优秀的参数组合。这样一个参数组合在全数据上表现优异的可能性是非常大的，同时也可能展现出比网格/随机搜索得出的参数更大的泛化能力。

- 局限性

然而这个过程当中会存在一个问题：子集越大时，子集与全数据集D的分布会越相似，但整个对半搜索算法在开头的时候，就用最小的子集筛掉了最多的参数组合。如果最初的子集与全数据集的分布差异巨大的化，在对半搜索开头的前几次迭代中，就可能筛掉许多对全数据集D有效的参数，因此对半网格搜索最初的子集一定不能太小。

在对半网格搜索过程中，子集的样本量时呈指数级增长：

```
n = 10
for i in range(10):
    print(i,n*2**i)

"""输出如下"""
0 10
1 20
2 40
3 80
4 160
5 320
6 640
7 1280
8 2560
9 5120
```

在初始子集样本量为10的前提下，7、8次迭代就会消耗掉2500+数据资源。在初始子集一定不能太小、且对半搜索的抽样是不放回抽样的大前提下，**整体数据的样本量必须要很大**。从经验来看，对半网格搜索在小型数据集上的表现往往不如随机网格搜索与普通网格搜索，但在大型数据集上（比如，样本量过w的数据集上），对半网格搜索展现出运算速度

上的巨大优势。

2.3.2 实现对半网格搜索

- sklearn中对半网格搜索的类如下

```
HalvingGridSearchCV(estimator # 调参对象，某评估器
                    , param_grid # 参数空间
                    , *
                    , factor=3 # 每轮迭代中新增的样本量的比例，同时也是每轮迭代后留下的参数组合的比例
                    , resource='n_samples'# 设置每轮迭代中增加的验证资源的类型
                    , max_resources='auto' # 在一次迭代中，允许被用来验证任意参数组合的最大样本量
                    , min_resources='exhaust' # 首次迭代时，用于验证参数组合的样本量r0
                    , aggressive_elimination=False # 是否以全部数被使用完成作为停止搜索的指标，如果不是，则采取措施
                    , cv=5 # 交叉验证的折数
                    , scoring=None # 评估指标，支持同时输出多个参数
                    , refit=True # 挑选评估指标和最佳参数，在完整数据集上进行训练
                    , error_score=nan
                    , return_train_score=True # 在交叉验证中是否显示训练集中参数得分
                    , random_state=None # 控制随机抽样数据集的随机性
                    , n_jobs=None # 设置工作时参与计算的线程数
                    , verbose=0 # 输出工作日志形式
                   )
```

全部参数如下所示：

Name	Description
estimator	调参对象，某评估器
param_grid	参数空间，可以是字典或者字典构成的列表
factor	每轮迭代中新增的样本量的比例，同时也是每轮迭代后留下的参数组合的比例
resource	设置每轮迭代中增加的验证资源的类型
max_resources	在一次迭代中，允许被用来验证任意参数组合的最大样本量
min_resources	首次迭代时，用于验证参数组合的样本量r0
aggressive_elimination	是否以全部数被使用完成作为停止搜索的指标，如果不是，则采取措施
cv	交叉验证的折数
scoring	评估指标，支持同时输出多个参数
refit	挑选评估指标和最佳参数，在完整数据集上进行训练
error_score	当网格搜索报错时返回结果，选择'raise'时将直接报错并中断训练过程
其他情况会显示警告信息后继续完成训练	
return_train_score	在交叉验证中是否显示训练集中参数得分
random_state	控制随机抽样数据集的随机性
n_jobs	设置工作时参与计算的线程数
verbose	输出工作日志形式

- **factor**

  每轮迭代中新增的样本量的比例，同时也是每轮迭代后留下的参数组合的比例。例如，当factor=2时，下一轮迭代的样本量会是上一轮的2倍，每次迭代后有1/2的参数组合被留下。如果factor=3时，下一轮迭代的样本量会是上一轮的3倍，每次迭代后有1/3的参数组合被留下。该参数通常取3时效果比较好。

- **resource**

  设置每轮迭代中增加的验证资源的类型，输入为字符串。默认是样本量，输入为"n_samples"，也可以是任意集成算法当中输入正整数的弱分类器，例如"n_estimators"或者"n_iteration"。

- **min_resource**

  首次迭代时，用于验证参数组合的样本量r0。可以输入正整数，或两种字符

串"smallest","exhaust"。
输入正整数n，表示首次迭代时使用n个样本。
输入"smallest"，则根据规则计算r0:

> 当资源类型是样本量时，对回归类算法，r0 = 交叉验证折数n_splits * 2
> 当资源类型是样本量时，对分类算法，r0 = 类别数量n_classes_ * 交叉验证折数n_splits * 2
> 当资源类型不是样本量时，等于1

输入"exhaust"，则根据迭代最后一轮的最大可用资源倒退r0。例如，factor=2, 样本量为1000时，一共迭代3次时，则最后一轮迭代的最大可用资源为1000，倒数第二轮为500，倒数第三轮（第一轮）为250。此时r0 = 250。"exhaust"模式下最有可能得到好的结果，不过计算量会略大，计算时间会略长。

现在，我们依然使用原来的参数空间：

```
import re
import sklearn
import numpy as np
import pandas as pd
import matplotlib as mlp
import matplotlib.pyplot as plt
import time
from sklearn.ensemble import RandomForestRegressor as RFR
from sklearn.experimental import enable_halving_search_cv
from sklearn.model_selection import KFold, HalvingGridSearchCV

data2 = pd.read_csv(r"./big_train.csv",index_col=0)
X = data2.iloc[:,:-1] # shape(29062, 80)
y = data2.iloc[:,-1]


param_grid_simple = {'n_estimators': [*range(5,100,5)]
                     , 'max_depth': [*range(25,36,2)]
                     , "max_features": ["log2","sqrt",16,32,64,"auto"]
                     }

#计算全域参数空间大小
len([*range(5,100,5)]) * len([*range(25,36,2)]) * 6

"""输出如下"""
684
```

哪一种停止条件会被触发？（训练集合shape(29062, 80)）

```
#1460个样本在factor=2, min_resource = 10的情况下可以迭代多久?
for i in range(100):
    if 10*2**i > 1460:
        break
    print(i,10*2**i)
```

```
#684种参数组合在factor=2的情况下可以迭代多久?
for i in range(100):
    if 684//2**i < 1:
        break
    print(i,684//2**i+1) #向上取整

"""输出如下"""
0 685
1 343
2 172
3 86
4 43
5 22
6 11
7 6
8 3
9 2
```

不难发现，当factor=2的时候，数据集不足的条件会先被触发，最多只能迭代7次。也就是说，最终我们将在6组参数中选择表现最好的一组参数，而不会一直让搜索持续直到找出唯一最优的参数。如果我们无论如何都希望能够找到唯一最后的参数，那我们可以使用下面的参数：

- **aggressive_elimination**

  输入布尔值，默认False。当数据总样本量较小，不足以支撑循环直到只剩下最后一组备选参数时，可以打开该参数。

> 参数设置为True时，会重复使用首次迭代时的样本量，直到剩下的数据足以支撑样本量的增加直到只剩下最后一组备选参数
> 参数设置为False时，以全部样本被用完作为搜索结束的指标

对于对半网格搜索应用来说，最困难的部分就是决定搜索本身复杂的参数组合。在调参时，如果我们希望参数空间中的备选组合都能够被充分验证，则迭代次数不能太少（例如，只迭代3次），因此factor不能太大。但如果factor太小，又会加大迭代次数，同时拉长整个搜索的运行时间。因此，我们一般考虑以下两个点：

1. min_resources的值不能太小，且迭代过程中使用尽量多的数据
2. 迭代完毕之后，剩余的验证参数组合不能太多，10~20之间即可接受

```
factor = 1.5
n_samples = X.shape[0]
n_splits = 5
min_resources = 500
space = 684

for i in range(100):
    if (min_resources*factor**i > n_samples) or (space/factor**i < 1):
        break
    print(i,"本轮迭代样本:{}".format(min_resources*factor**i)
```

```
            ,"本轮验证参数组合:{}".format(space//factor**i + 1))

"""输出如下"""
0 本轮迭代样本:500.0 本轮验证参数组合:685.0
1 本轮迭代样本:750.0 本轮验证参数组合:457.0
2 本轮迭代样本:1125.0 本轮验证参数组合:305.0
3 本轮迭代样本:1687.5 本轮验证参数组合:203.0
4 本轮迭代样本:2531.25 本轮验证参数组合:136.0
5 本轮迭代样本:3796.875 本轮验证参数组合:91.0
6 本轮迭代样本:5695.3125 本轮验证参数组合:61.0
7 本轮迭代样本:8542.96875 本轮验证参数组合:41.0
8 本轮迭代样本:12814.453125 本轮验证参数组合:27.0
9 本轮迭代样本:19221.6796875 本轮验证参数组合:18.0
10 本轮迭代样本:28832.51953125 本轮验证参数组合:12.0
```

- 建立回归器、交叉验证、定义随机搜索

```
#建立回归器、交叉验证
reg = RFR(random_state=1412,verbose=True,n_jobs=12)
cv = KFold(n_splits=5,shuffle=True,random_state=1412)

#定义随机搜索
search = HalvingGridSearchCV(estimator=reg
                             ,param_grid=param_grid_simple
                             ,factor=1.5
                             ,min_resources=500
                             ,scoring = "neg_mean_squared_error"
                             ,verbose = True
                             ,random_state=1412
                             ,cv = cv
                             ,n_jobs=12)
```

```
#训练随机搜索评估器
#=====【TIME WARNING: 5~10min】=====#
start = time.time()
search.fit(X,y)
print(time.time() - start)

"""输出如下"""
2678.4405131340027
```

```
2678/60

44分钟左右
```

```
#查看最佳评估器
search.best_estimator_

"""输出如下"""
RandomForestRegressor(max_depth=27
                      , max_features=32
                      , n_estimators=95
                      , n_jobs=12
                      , random_state=1412
                      , verbose=True)
```

```
#查看rmse
abs(search.best_score_)**0.5

"""输出如下"""
750.4062902073151
```

- 以随机网格搜索作为对比，我们来看看随机网格搜索的结果：

```
param_grid_simple = {'n_estimators': [*range(5,100,5)]
                     , 'max_depth': [*range(25,36,2)]
                     , "max_features": ["log2","sqrt",16,32,64,"auto"]
                     }

reg = RFR(random_state=100
          ,verbose=True
          ,n_jobs=12
         )
cv = KFold(n_splits=5
           ,shuffle=True
           ,random_state=100
          )

#定义随机搜索
search = RandomizedSearchCV(estimator=reg
                            ,param_distributions=param_grid_simple
                            ,n_iter = 360 #还是使用360这个搜索次数
                            ,scoring = "neg_mean_squared_error"
                            ,verbose = True
                            ,cv = cv
                            ,n_jobs=12
                           )
#训练随机搜索评估器
#=====【TIME WARNING: 5~10min】=====#
start = time.time()
search.fit(X,y)
print(time.time() - start)
```

```

```

```python
#查看最佳评估器
search.best_estimator_

"""输出如下"""
```

```python
#查看最终评估指标rmse
abs(search.best_score_)**0.5

"""输出如下"""
```

HPO方法	随机搜索	Halving搜索
搜索空间/全域空间	360/684	684/684
搜索时间	25.3mins	19mins
最优分数（RMSE）	751.988	750.406

3 三、bayes优化

3.1 parameter tuning with hyperopt

3.1.1 abstract

> https://blog.csdn.net/weixin_41048094/article/details/91347454?spm=1001.2101.3001.6650.1&utm_medium=distribute.pc_relevant.none-task-blog-2%7Edefault%7ECTRLIST%7ERate-1.pc_relevant_antiscanv2&depth_1-utm_source=distribute.pc_relevant.none-task-blog-2%7Edefault%7ECTRLIST%7ERate-1.pc_relevant_antiscanv2&utm_relevant_index=2

- This post will cover a few things needed to quickly implement a fast, principled method for machine learning model parameter tuning. There are two common methods of parameter tuning: grid search and random search. Each have their pros and cons. Grid search is slow but effective at searching the whole search space, while random search is fast, but could miss important points in the search space. Luckily, a third option exists: Bayesian optimization. In this post, we will focus on one implementation of Bayesian optimization, a Python module called `hyperopt`.

  这篇文章将介绍快速实现机器学习模型参数调整的快速、有原则的方法所需的一些内容。 常用的参数调优方法有两种：网格搜索和随机搜索。 每个都有其优点和缺点。 网格搜索很慢，但在搜索整个搜索空间时很有效，而随机搜索很快，但可能会错过搜索空间中的重要点。 幸运的是，存在第三种选择：贝叶斯优化。 在这篇文章中，我们将重点介绍贝叶斯优化的一种实现，即一个名为 hyperopt 的 Python 模块。

- Using Bayesian optimization for parameter tuning allows us to obtain the best parameters for a given model, e.g., logistic regression. This also allows us to perform optimal model selection. Typically, a machine learning engineer or data scientist will perform some form of manual parameter tuning (grid search or random search) for a few models - like decision tree, support vector machine, and k nearest neighbors - then compare the accuracy scores and select the best one for use. This method has the possibility of comparing sub-optimal models. Maybe the data scientist found the optimal parameters for the decision tree, but missed the optimal parameters for SVM. This means their model comparison was flawed. K nearest neighbors may beat SVM every time if the SVM parameters are poorly tuned. Bayesian optimization allow the data scientist to find the *best* parameters for all models, and therefore compare the best models. This results in better model selection, because you are comparing the *best* k nearest neighbors to the *best* decision tree. Only in this way can you do model selection with high confidence, assured that the actual best model is selected and used.

  使用贝叶斯优化进行参数调整可以让我们获得给定模型的最佳参数，例如逻辑回归。这也使我们能够执行最佳模型选择。通常，机器学习工程师或数据科学家会对一些模型（如决策树、支持向量机和 k 个最近邻）执行某种形式的手动参数调整（网格搜索或随机搜索），然后比较准确度得分并选择最好的使用。这种方法有可能比较次优模型。也许数据科学家找到了决策树的最佳参数，但错过了 SVM 的最佳参数。这意味着他们的模型比较存在缺陷。如果 SVM 参数调整不佳，K 最近邻可能每次都击败 SVM。贝叶斯优化允许数据科

学家找到所有模型的最佳参数，从而比较最佳模型。这会导致更好的模型选择，因为您正在将最佳 k 最近邻与最佳决策树进行比较。只有这样，您才能高枕无忧地进行模型选择，确保选择和使用实际的最佳模型。

- Topics covered are in this post are:
    1. Objective functions
    2. Search spaces
    3. Storing evaluation trials
    4. Visualization
    5. Full example on a classic dataset: Iris

  To use the code below, you must install `hyperopt` and `pymongo`.

  这篇文章中涉及的主题是：
    1. 目标函数
    2. 搜索空间
    3. 存储评估试验
    4. 可视化
    5. 经典数据集的完整示例：鸢尾花

  要使用下面的代码，您必须安装 hyperopt 和 pymongo。

3.1.2 Objective Functions - A Motivating Example

- 目标函数——一个启发性的例子*
- Suppose you have a function defined over some range, and you want to minimize it. That is, you want to find the input value that result in the lowest output value. The trivial example below finds the value of `x` that minimizes a linear function

  假设您在某个范围内定义了一个函数，并且您希望最小化它。 也就是说，您想要找到导致最低输出值的输入值。 下面的简单示例找到使线性函数最小化的 x 值。

```
from hyperopt import fmin, tpe, hp

best = fmin(fn=lambda x: x   # fn接收一个最小化函数
            ,space=hp.uniform('x' # 参数别名
                              , 0 # 下限
                              , 1 # 上限
                             ) # 指定搜索空间
            ,algo=tpe.suggest # 采用搜索算法
            ,max_evals=100 # 指定fmin函数将执行的最大评估次数
           )

print(best)


"""输出如下"""
{'x': 0.0002258846235372123}
```

- Let's break this down.

  The function fmin first takes a function to minimize, denoted fn, which we here specify with an anonymous function lambda x: x. This function could be any valid value-returning function, such as mean absolute error in regression.

  函数 fmin 首先采用一个最小化函数，记为 fn，我们在这里用匿名函数 lambda x: x 指定它。该函数可以是任何有效的值返回函数，例如回归中的平均绝对误差。

  The next parameter specifies the search space, and in this example it is the continuous range of numbers between 0 and 1, specified by hp.uniform('x', 0, 1). hp.uniform is a built-in hyperopt function that takes three parameters: the name, x, and the lower and upper bound of the range, 0 and 1.

  下一个函数指定搜索空间，在此示例中他是由 `hp.uniform('x',0,1)` 指定的0到1之间的连续数字范围。hp.uniform 是一个内置的 hyperopt 函数，它接受三个参数：名称 x 以及范围的下限和上限 0 和 1。

  The parameter algo takes a search algorithm, in this case tpe which stands for tree of Parzen estimators. This topic is beyond the scope of this blog post, but the mathochistic reader may peruse this for details. The algo parameter can also be set to hyperopt.random, but we do not cover that here as it is widely known search strategy. However, in a future post, we can.

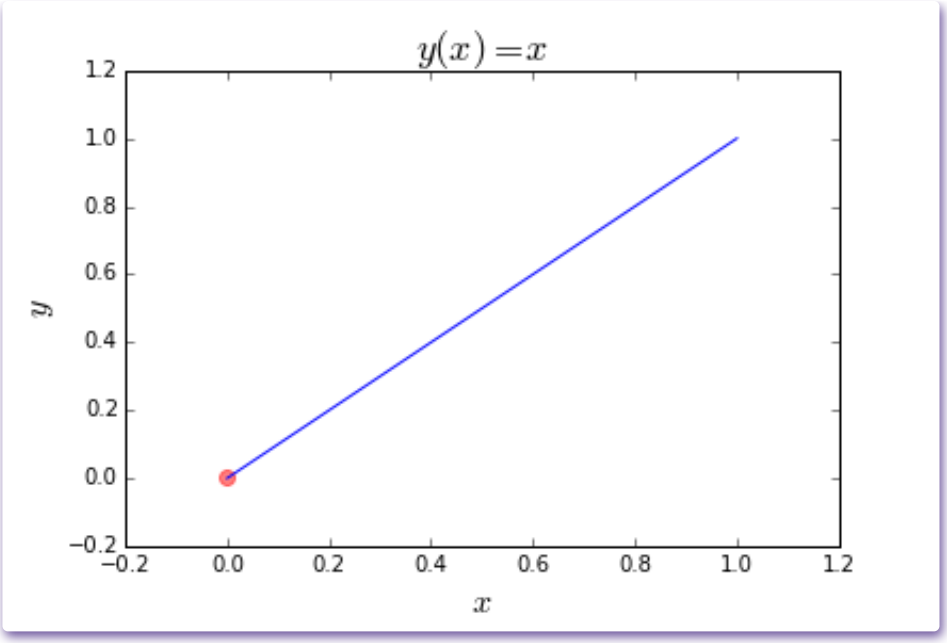
  参数 algo 采用搜索算法，在本例中 tpe 代表 Parzen 估计器树。这个话题超出了这篇博文的范围，但是有数学背景的同学可以细读**这篇**文章。 algo 参数也可以设置为 hyperopt.random，但我们不在这里讨论它，因为它是众所周知的搜索策略。但是在未来的文章中我们可能会涉及。

  Finally, we specify the maximum number of evaluations max_evals the fmin function will perform. This fmin function returns a python dictionary of values.An example of the output for the function above is {'x': 0.0002258846235372123}.

  最后，我们指定 fmin 函数将执行的最大评估次数 max_evals。 这个 fmin 函数返回一个 Python 值字典。上述函数的输出示例是{'x': 0.0002258846235372123}。

  Here is the plot of the function. The red dot is the point we are trying to find.

  这个是函数的图。红点是我们试图找到的点。



3.1.3 More Complicated Examples

- 一个更复杂的例子
- Here is a more complicated objective function: `lambda x: (x-1)**2`. This time we are trying to minimize a quadratic equation `y(x) = (x-1)**2`. So we alter the search space to include what we know to be the optimal value (`x=1`) plus some sub-optimal ranges on either side: `hp.uniform('x', -2, 2)`.

  Now we have:

  这是一个更复杂的目标函数：lambda x: (x-1)**2。 **这次我们试图最小化一个二次方程 y(x) = (x-1)**2。 所以我们改变搜索空间以包括我们知道的最优值 (x=1) 加上两边的一些次优范围：hp.uniform('x', -2, 2)。

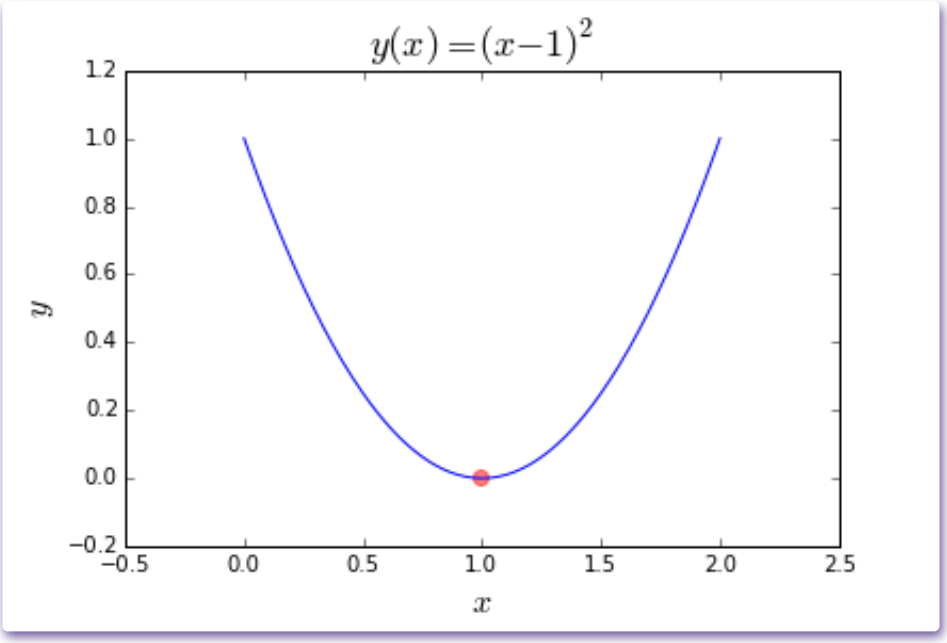
  现在我们有：

```
from hyperopt import fmin,tpe,hp

best = fmin(fn=lambda x: (x-1)**2
            ,space=hp.uniform('x', -2, 2)
            ,algo=tpe.suggest
            ,max_evals=100
           )
print(best)
```

  The output should look something like this:

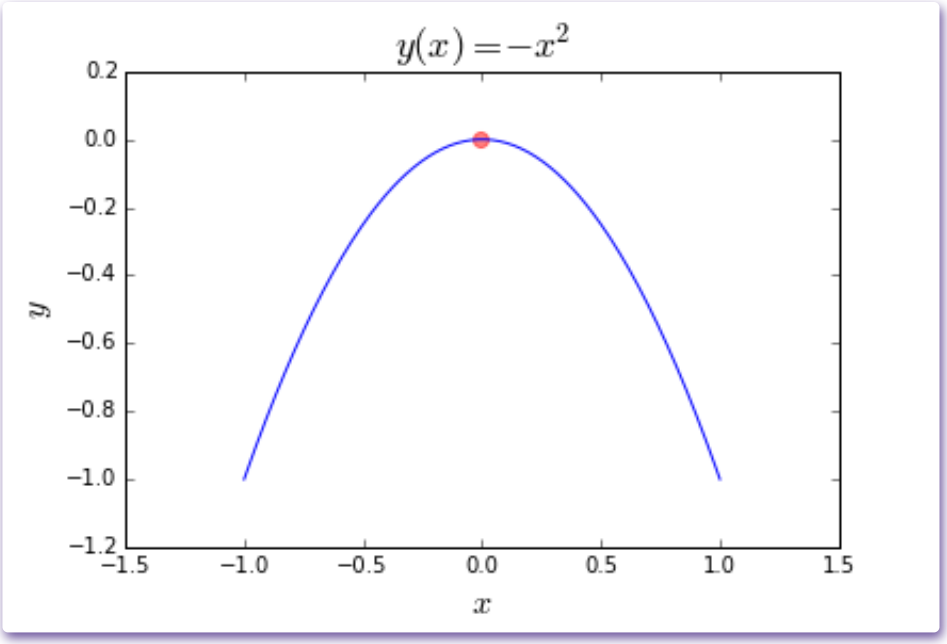
```
"""输出如下"""
{'x': 0.9958194793700939}
```

  Here is the plot.



- Instead of minimizing an objective function, maybe we want to maximize it. To to this we need only return the negative of the function. For example, we could have a function `y(x) = -(x**2)`:

  有时候可能我们并不是想着最小化一个函数，而是最大化它。为此，我们只需要返回函数的负数即可。例如，我们可以有一个函数 `y(x) = -(x**2)`：

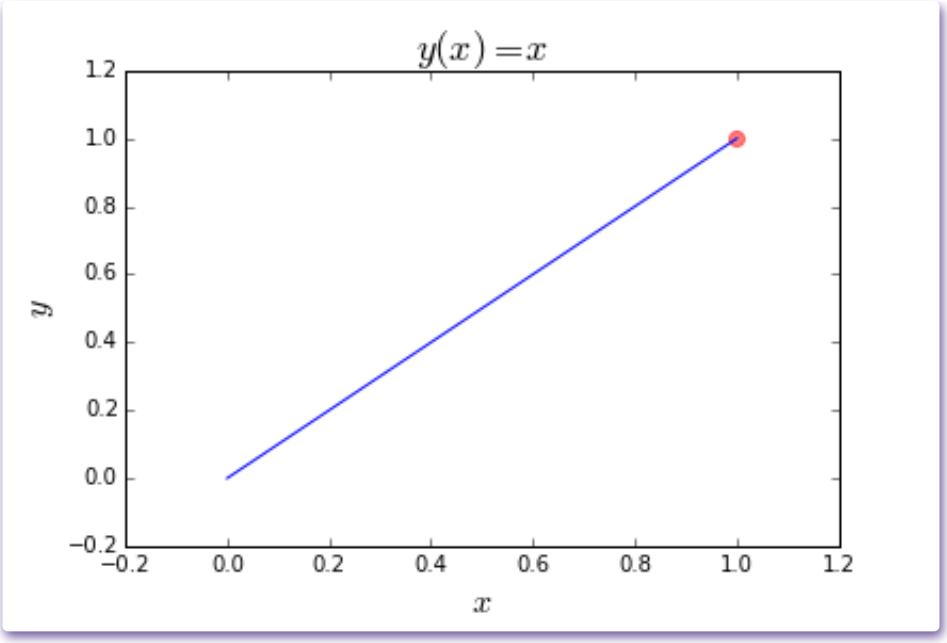


  How could we go about solving this? We just take the objective function `lambda x: -(x**2)` and return the negative, giving `lambda x: -1*-(x**2)` or just `lambda x: (x**2)`.

  我们该如何解决这个问题？ 我们只取目标函数 lambda x: -(x**2) **并返回负数，给出 lambda x: -1*-(x**2)** 或只是 lambda x: (x**2)。

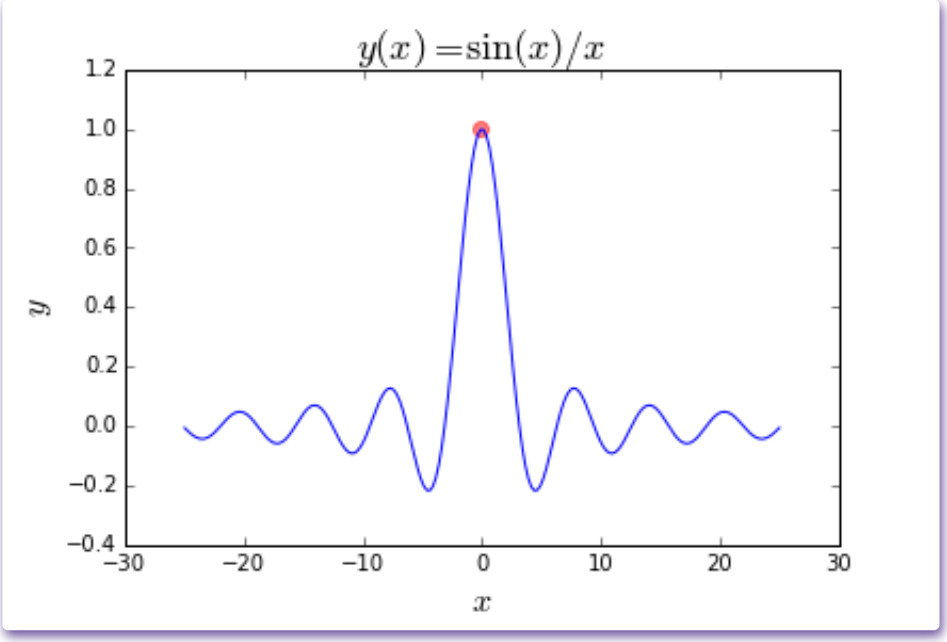
- Here is one similar to example 1, but instead of minimizing, we are trying to maximize.

  这是一个类似于示例 1 的示例，但我们不是最小化，而是试图最大化y=x。



- Here is a function with many (infinitely many given an infinite range) local minima, which we are also trying to maximize:

  这是一个具有许多（在无限范围内无限多）局部最小值的函数，我们也试图将其最大化：



3.1.4 Search Spaces

- 搜索空间
- The `hyperopt` module includes a few handy functions to specify ranges for input parameters. We have already seen `hp.uniform`. Initially, these are stochastic search spaces, but as `hyperopt` learns more (as it gets more feedback from the objective function), it adapts and samples different parts of the initial search space that it thinks will give it the most meaningful feedback.

  The following will be used in this post:

  1. `hp.choice(label, options)` where `options` should be a python list or tuple.

2. `hp.normal(label, mu, sigma)` where `mu` and `sigma` are the mean and standard deviation, respectively.
3. `hp.uniform(label, low, high)` where `low` and `high` are the lower and upper bounds on the range.

hyperopt 模块包括一些方便的函数来指定输入参数的范围。 我们已经看到了 hp.uniform。最初，这些是随机搜索空间，但随着 hyperopt 学习得更多（因为它从目标函数获得更多反馈），它会适应并采样它认为会给它最有意义的反馈的初始搜索空间的不同部分。

以下内容将在本文中使用：

1. `hp.choice(label, options)` 其中 `options` 应是 python 列表或元组。
2. `hp.normal(label, mu, sigma)` 其中 `mu` 和 `sigma` 分别是均值和标准差。
3. `hp.uniform(label, low, high)` 其中 `low` 和 `high` 是范围的下限和上限。

- Others are available, such as `hp.normal`, `hp.lognormal`, `hp.quniform`, but we will not use them here.

  To see some draws from the search space, we should import another function, and define the search space.

  其他可用的相关函数，例如 hp.normal、hp.lognormal、hp.quniform，但我们不会在这里使用它们。

  要查看搜索空间的一些绘制，我们应该导入另一个函数，并定义搜索空间。

```
import hyperopt.pyll.stochastic

space = {'x': hp.uniform('x', 0, 1)
         ,'y': hp.normal('y', 0, 1)
         ,'name': hp.choice('name', ['alice', 'bob']),
}

print(hyperopt.pyll.stochastic.sample(space))
```

  An example output is:

```
"""输出如下"""
{'name': 'alice', 'x': 0.7102704161240886, 'y': -1.102497756563188}
```

3.1.5 Capturing Info with Trials

- 通过 `Trials` 捕捉信息
- It would be nice to see exactly what is happening inside the `hyperopt` black box. The `Trials` object allows us to do just that. We need only import a few more items.

  如果能看到 `hyperopt` 黑匣子内发生了什么是极好的。 `Trials` 对象使我们能够做到这一点。我们只需要再导入一些东西。

```
from hyperopt import fmin, tpe, hp, STATUS_OK, Trials


```

```python
fspace = {'x': hp.uniform('x', -5, 5)}

def f(params):
    x = params['x']
    val = x**2
    return {'loss': val, 'status': STATUS_OK}

trials = Trials()
best = fmin(fn=f
            , space=fspace
            , algo=tpe.suggest
            , max_evals=50
            , trials=trials
           )

print('best:\n', best)

print('trials:')
for trial in trials.trials[:2]:
    print(trial)
```

The `STATUS_OK` and `Trials` imports are new. The `Trials` object allows us to store info at each time step they are stored. We can then print them out and see what the evaluations of the function were for a given parameter at a given time step.

`STATUS_OK` 和 `Trials` 是新导入的。 `Trials` 对象允许我们在每个时间步存储信息。然后我们可以将它们打印出来，并在给定的时间步查看给定参数的函数评估值。

Here is an example output of the code above:

```
"""输出如下"""
"""
需要注意的点
1. tid 是从0 到 max_evals-1
2. x(params)存储在misc中vals里
3. loss在result中（实际就是fmin函数返回的值）
"""

best:
 {'x': -0.0008001691326020577}
trials:
{
      'state': 2
    , 'tid': 0
    , 'spec': None
    , 'result': {'loss': 0.08079044392675966, 'status': 'ok'}
    , 'misc': {'tid': 0, 'cmd': ('domain_attachment', 'FMinIter_Domain'), 'workdir': None, 'idxs': {'x': [0]}, 'vals': {'x': [-0.28423659849984073]}}
    , 'exp_key': None
    , 'owner': None
    , 'version': 0
```

```
        , 'book_time': datetime.datetime(2022, 4, 12, 14, 5, 15, 572000)
        , 'refresh_time': datetime.datetime(2022, 4, 12, 14, 5, 15, 572000)
    }



    {
          'state': 2
        , 'tid': 1
        , 'spec': None
        , 'result': {'loss': 15.862399717087962, 'status': 'ok'}
        , 'misc': {'tid': 1, 'cmd': ('domain_attachment', 'FMinIter_Domain'), 'workdir': None, 'idxs': {'x': [1]}, 'vals': {'x': [3.982762824609063]}}
        , 'exp_key': None
        , 'owner': None
        , 'version': 0
        , 'book_time': datetime.datetime(2022, 4, 12, 14, 5, 15, 573000)
        , 'refresh_time': datetime.datetime(2022, 4, 12, 14, 5, 15, 573000)
    }
```

- The trials object stores data as a `BSON` object, which works just like a `JSON` object. `BSON` is from the `pymongo` module. We will not discuss the details here, but there are advanced options for `hyperopt` that require distributed computing using `MongoDB`, hence the `pymongo` import.

  Back to the output above. The `'tid'` is the time id, that is, the time step, which goes from `0` to `max_evals-1`. It increases by one each iteration. `'x'` is in the `'vals'` key, which is where your parameters are stored for each iteration. `'loss'` is in the `'result'` key, which gives us the value for our objective function at that iteration.

  Let's look at this in another way.

  `Trials` 对象将数据存储为 `BSON` 对象，其工作方式与 `JSON对象` 相同。 `BSON` 来自 `pymongo` 模块。我们不会在这里讨论细节，但是 hyperopt 有一些高级选项需要使用 MongoDB 进行分布式计算，因此需要导入 pymongo 。

  'tid' 是时间 id，即时间步长，从 0 到 max_evals-1。它随着迭代次数递增。 `'x'` 是键 `'vals'` 的值，其中存储的是每次迭代参数的值。 `'loss'` 是键 `'result'` 的值，其给出了该次迭代目标函数的值。

- Let's look at this in another way.、

3.1.6 Visualization

- We'll go over two types of visualizations here:

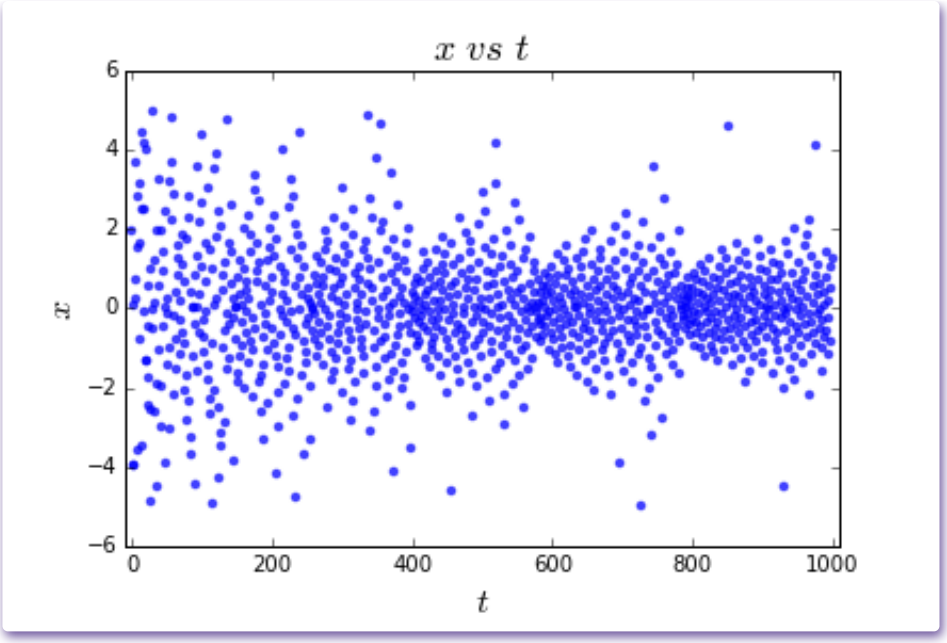
  1. val vs. time

2. loss vs. val.

First, val vs. time. Below is the code and sample output for plotting the `trials.trials` data described above.

我们将在这里讨论两种类型的可视化：val vs. time，以及 loss vs. val。 首先，val vs time。下面是用于绘制上述 trial.trials 数据的代码和示例输出(为了效果，max_evals自己调到了5000)。

```
from hyperopt import fmin, tpe, hp, STATUS_OK, Trials
from matplotlib import pyplot as plt

fspace = {'x': hp.uniform('x', -5, 5)}

def f(params):
    x = params['x']
    val = x**2
    return {'loss': val, 'status': STATUS_OK}

trials = Trials()
best = fmin(fn=f
            , space=fspace
            , algo=tpe.suggest
            , max_evals=5000
            , trials=trials
           )

f, ax = plt.subplots(1)
xs = [t['tid'] for t in trials.trials] # tid 是从0 到 max_evals - 1
ys = [t['misc']['vals']['x'] for t in trials.trials] # 存储的是x (params)
ax.set_xlim(xs[0]-10, xs[-1]+10)
ax.scatter(xs, ys, s=20, linewidth=0.01, alpha=0.75)
ax.set_title('$x$ $vs$ $t$ ', fontsize=18)
ax.set_xlabel('$t$', fontsize=16)
ax.set_ylabel('$x$', fontsize=16)
plt.show()
```



We can see that initially the algorithm picks values from the whole range equally (uniformly), but as time goes on and more is learned about the parameter's effect on the objective function, the algorithm focuses more and more on areas in which it thinks it will gain the most - the range close to zero. It still explores the whole solution space, but less frequently.

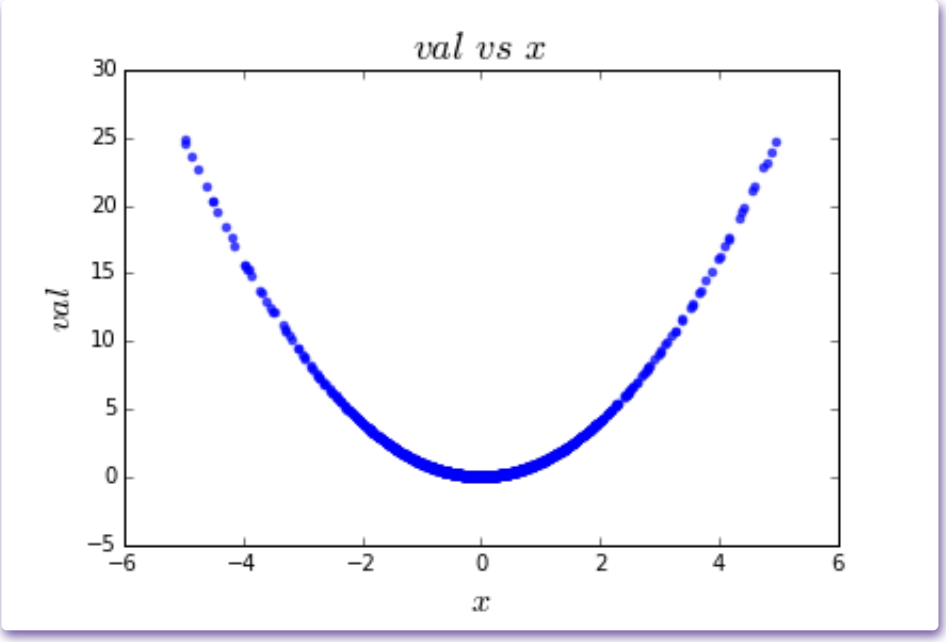
Now let's look at the plot of loss vs. val.

我们可以看到，最初算法从整个范围中均匀地选择值，但随着时间的推移以及参数对目标函数的影响了解越来越多，该算法越来越聚焦于它认为会取得最大收益的区域-一个接近零的范围。它仍然探索整个解空间，但频率有所下降。

现在让我们看看loss vs. val的图。

```
from hyperopt import fmin, tpe, hp, STATUS_OK, Trials
from matplotlib import pyplot as plt

fspace = {'x': hp.uniform('x', -5, 5)}

def f(params):
    x = params['x']
    val = x**2
    return {'loss': val, 'status': STATUS_OK}

trials = Trials()
best = fmin(fn=f
            , space=fspace
            , algo=tpe.suggest
            , max_evals=5000
            , trials=trials
           )

f, ax = plt.subplots(1)
xs = [t['misc']['vals']['x'] for t in trials.trials]
ys = [t['result']['loss'] for t in trials.trials]
ax.scatter(xs, ys, s=20, linewidth=0.01, alpha=0.75)
ax.set_title('$val$ $vs$ $x$ ', fontsize=18)
```

```
ax.set_xlabel('$x$', fontsize=16)
ax.set_ylabel('$val$', fontsize=16)
plt.show()
```



This gives us what we expect, since the function `y(x) = x**2` is deterministic.

To wrap up, let's try a more complicated example, with more randomness and more parameters.

这给了我们预期的结果，因为函数 y(x) = x**2 是确定性的。

最后，让我们尝试一个更复杂的例子，它有更多的随机性和更多的参数。

3.1.7 The Iris Dataset

- In this section, we'll walk through 4 full examples of using `hyperopt` for parameter tuning on a classic dataset, Iris. We will cover K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Decision Trees, and Random Forests. Note that since we are trying to maximize the cross-validation accuracy (`acc` in the code below), we must negate this value for `hyperopt`, since `hyperopt` only knows how to minimize a function. Minimizing a function `f` is the same as maximizing the negative of `f`.

  For this task, we'll use the classic Iris data set, and do some supervised machine learning. There are 4 input features, and three output classes. The data are labeled as belonging to class 0, 1, or 2, which map to different kinds of Iris flower. The input has 4 columns: sepal length, sepal width, petal length, and pedal width. Units of the input are centimeters. We will use these 4 features to learn a model that predicts one of three output classes. Since the data is provided by `sklearn`, it has a nice DESCR attribute that provides details on the data set. Try the following for more details.

  在本节中，我们将介绍 4 个使用 hyperopt 对经典数据集 Iris 进行参数调整的完整示例。我们将介绍 K 近邻 (KNN)、支持向量机 (SVM)、决策树和随机森林。请注意，由于我们试图最大化交叉验证的准确性（acc 在下面的代码中），我们必须对 hyperopt 取反，因为 hyperopt 只知道如何最小化函数。最小化函数 f 与最大化 f 的负值相同。

对于这个任务，我们将使用经典的 Iris 数据集，并进行一些监督机器学习。有 4 个输入特征和三个输出类。数据被标记为属于 0、1 或 2 类，它们映射到不同种类的鸢尾花。输入有 4 列：萼片长度、萼片宽度、花瓣长度和踏板宽度。输入的单位是厘米。我们将使用这 4 个特征来学习预测三个输出类别之一的模型。由于数据是由 sklearn 提供的，它有一个很好的 DESCR 属性，可以提供数据集的详细信息。请尝试以下操作以获取更多详细信息。

```
from sklearn.datasets import load_iris

iris = load_iris()
print(iris.feature_names) # input names
print(iris.target_names) # output names
print(iris.DESCR) # everything else
```

Let's get to know the data a little better through visualization of the features and classes, using the code below. Don't forget to `pip install seaborn` if you have not already.

让我们通过使用下面的代码可视化特征和类来更好地了解数据。如果你还没安装别忘了先执行 `pip install searborn` 。

```
import seaborn as sns
import pandas as pd

sns.set(style="whitegrid", palette="husl")

iris = sns.load_dataset("iris")
print(iris.head())

iris = pd.melt(iris, "species", var_name="measurement")
print(iris.head())

f, ax = plt.subplots(1, figsize=(15,10))
sns.stripplot(x="measurement"
              , y="value"
              , hue="species"
              , data=iris
              , jitter=True
              , edgecolor="white"
              , ax=ax
             )
```

Here is the plot:

3.1.8 K-Nearest Neighbors

- We now apply `hyperopt` to finding the best parameters to a K-Nearest Neighbor (KNN) machine learning model. The KNN model classifies a data point from the test set based on majority class of the k nearest data points in the training data set. More information on this algorithm can he found **here**.
  The code below incorporates everything we have covered.

  我们现在将使用 `hyperopt` 来找到 K近邻（KNN）机器学习模型的最佳参数。KNN 模型是基于训练数据集中 k 个最近数据点的大多数类别对来自测试集的数据点进行分类。关于这个算法的更多信息可以参考 **这里** 。下面的代码结合了我们所涵盖的一切。

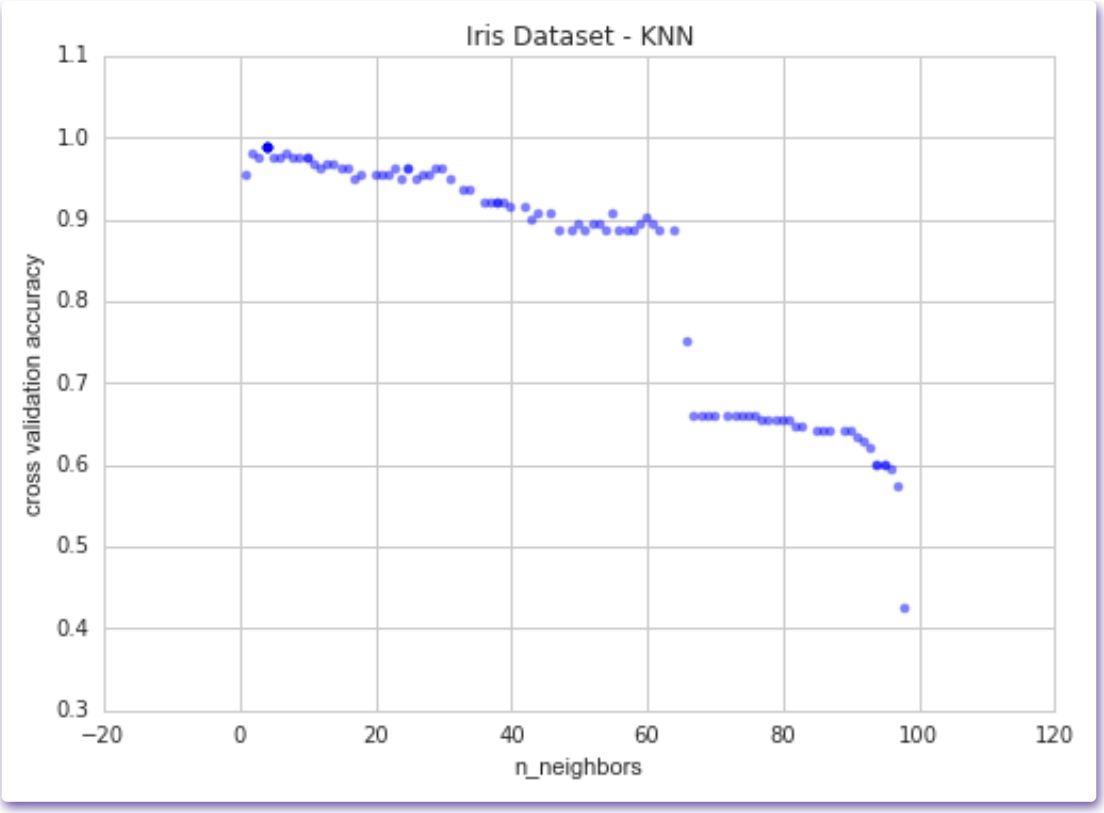
```
from sklearn import datasets
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier


iris = datasets.load_iris()

X = iris.data
y = iris.target

space4knn = {
    'n_neighbors': hp.choice('n_neighbors', range(1,100))
}

def f(params):
    clf = KNeighborsClassifier(**params)
    acc = cross_val_score(clf, X, y).mean()
```

```

    return {'loss': -acc, 'status': STATUS_OK}

trials = Trials()

best = fmin(f
            , space4knn
            , algo=tpe.suggest
            , max_evals=100
            , trials=trials
           )

print('best:\n',best)

"""输出如下"""
best:
 {'n_neighbors': 9}
```

Now let's see the plot of the output. The y axis is the cross validation score, and the x axis is the `k` value in k-nearest-neighbors. Here is the code and its image:

现在让我们看看输出图。 y 轴是交叉验证分数，x 轴是knn中的 k 值。 以下是代码及其图像：

```
f, ax = plt.subplots(1)#, figsize=(10,10))
xs = [t['misc']['vals']['n_neighbors'] for t in trials.trials]
ys = [-t['result']['loss'] for t in trials.trials]
ax.scatter(xs, ys, s=20, linewidth=0.01, alpha=0.5)
ax.set_title('Iris Dataset - KNN', fontsize=18)
ax.set_xlabel('n_neighbors', fontsize=12)
ax.set_ylabel('cross validation accuracy', fontsize=12)
```



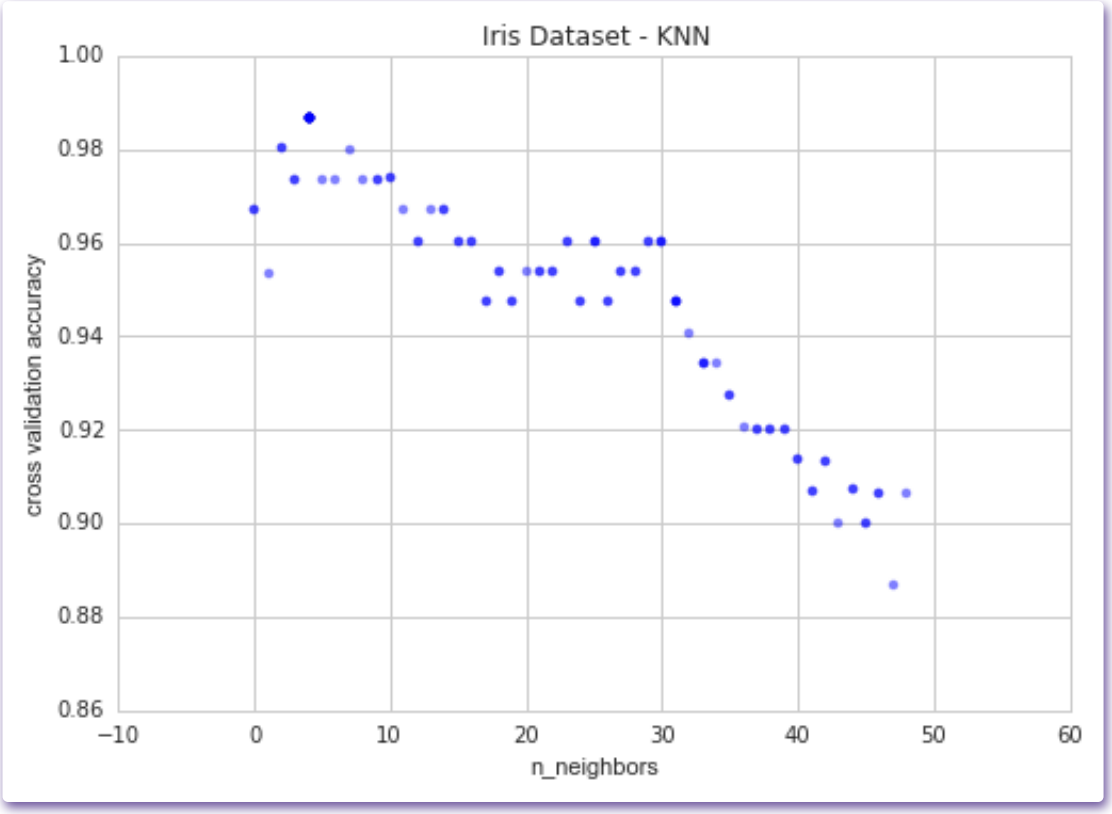
After `k` is greater than 69, the accuracy drops precipitously. This is due to the number of each class in the dataset. There are only 50 instances of each of the three classes. So let's drill down by limiting the values of `'n_neighbors'` to smaller values.

k大于63后，准确率急剧下降。 这是由于数据集中每个类的数量。 这三个类中的每一个只有 50 个实例。 因此，让我们通过将“n_neighbors”的值限制为更小的值来深入研究。

```
from sklearn import datasets
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier


iris = datasets.load_iris()

X = iris.data
y = iris.target

space4knn = {
    'n_neighbors': hp.choice('n_neighbors', range(1,50))
}

def f(params):
    clf = KNeighborsClassifier(**params)
    acc = cross_val_score(clf, X, y).mean()

    return {'loss': -acc, 'status': STATUS_OK}

trials = Trials()

best = fmin(f
            , space4knn
```

```
                , algo=tpe.suggest
                , max_evals=100
                , trials=trials
               )

print('best:\n',best)

f, ax = plt.subplots(1)#, figsize=(10,10))
xs = [t['misc']['vals']['n_neighbors'] for t in trials.trials]
ys = [-t['result']['loss'] for t in trials.trials]
ax.scatter(xs, ys, s=20, linewidth=0.01, alpha=0.5)
ax.set_title('Iris Dataset - KNN', fontsize=18)
ax.set_xlabel('n_neighbors', fontsize=12)
ax.set_ylabel('cross validation accuracy', fontsize=12)

"""输出如下"""
{'n_neighbors': 5}
```

Here is what we get when we run the same code for visualization:



Now we can see clearly that there is a best value for `k`, at `k` = 5

The model above does not do any preprocessing. So let's normalize and scale our features and see if that helps. Use this code:

现在我们可以清楚地看到 k 有一个最佳值，在 k = 4。

上面的模型不做任何预处理。 所以让我们使用以下代码规范化和扩展我们的特征，看看是否有帮助。

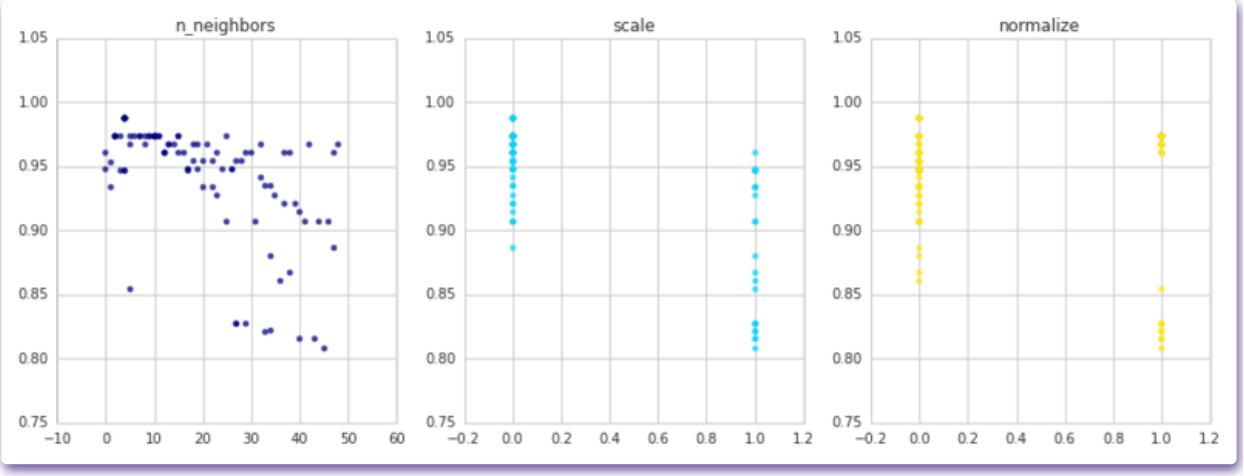
```
# now with scaling as an option
from sklearn import datasets
from sklearn.model_selection import cross_val_score
```

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
import numpy as np

iris = datasets.load_iris()
X = iris.data
y = iris.target

space4knn = {
    'n_neighbors': hp.choice('n_neighbors', range(1,50)),
    'scale': hp.choice('scale', [0, 1]),
    'normalize': hp.choice('normalize', [0, 1])
}

def f(params):
    X_ = X[:]

    if 'normalize' in params:
        if params['normalize'] == 1:
            normalize = MinMaxScaler()
            X_ = normalize.fit_transform(X_)
    del params['normalize']

    if 'scale' in params:
        if params['scale'] == 1:
            scale = StandardScaler()
            X_ = scale.fit_transform(X_)
    del params['scale']

    clf = KNeighborsClassifier(**params)
    acc = cross_val_score(clf, X_, y).mean()

    return {'loss': -acc, 'status': STATUS_OK}

trials = Trials()
best = fmin(f
            , space=space4knn
            , algo=tpe.suggest
            , max_evals=100
            , trials=trials
           )

print('best:\n',best)

"""输出如下"""
{'n_neighbors': 5, 'normalize': 0, 'scale': 0}
```

And plot the parameters like this:

```python
parameters = ['n_neighbors', 'scale', 'normalize']
cols = len(parameters)
```

```
f, axes = plt.subplots(nrows=1, ncols=cols, figsize=(15,5))
cmap = plt.cm.jet
for i, val in enumerate(parameters):
    xs = np.array([t['misc']['vals'][val] for t in trials.trials]).ravel()
    ys = [-t['result']['loss'] for t in trials.trials]
    xs, ys = zip(*sorted(zip(xs, ys)))
    ys = np.array(ys)
    axes[i].scatter(xs
                    , ys
                    , s=20
                    , linewidth=0.01
                    , alpha=0.75
                    , c=cmap(float(i)/len(parameters))
                   )
    axes[i].set_title(val)
```



We see that scaling and/or normalizing the data does not improve predictive accuracy. The best value of `k` remains 5, which results in 98.6 % accuracy.

So this is great for parameter tuning a simple model, KNN. Let's see what we can do with Support Vector Machines (SVM).

我们看到缩放和/或规范化数据并不能提高预测准确性。 k 的最佳值仍然是 5，这导致 98.6% 的准确度。

所以，这对于调整简单模型 KNN 的参数非常有用。 让我们看看我们可以用支持向量机 (SVM) 做什么。

3.1.9 Support Vector Machines (SVM)

- Since this is a classification task, we'll use `sklearn`'s `SVC` class. Here is the code:

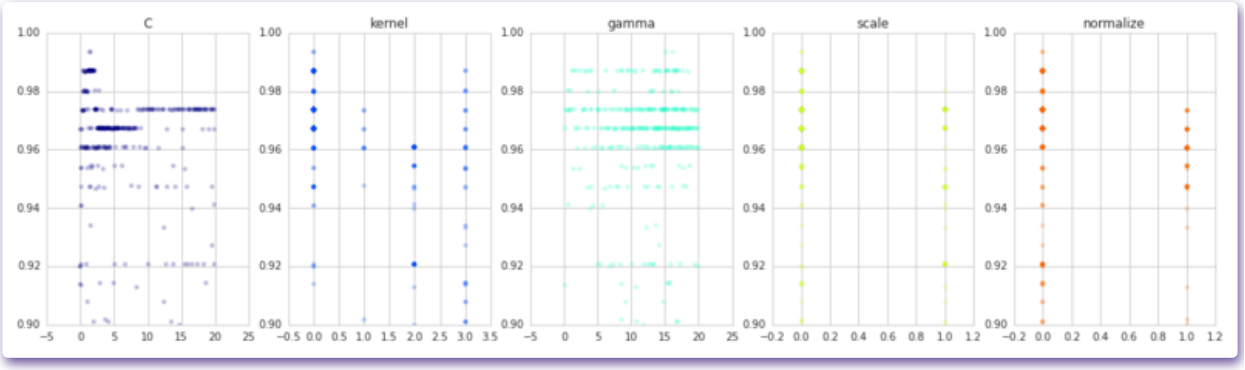
  由于这是一个分类任务，我们将使用 sklearn 的 SVC 类。

```
from sklearn import datasets
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
```

```python
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
import numpy as np


iris = datasets.load_iris()
X = iris.data
y = iris.target

space4svm = {
    'C': hp.uniform('C', 0, 20),
    'kernel': hp.choice('kernel', ['linear', 'sigmoid', 'poly', 'rbf']),
    'gamma': hp.uniform('gamma', 0, 20),
    'scale': hp.choice('scale', [0, 1]),
    'normalize': hp.choice('normalize', [0, 1])
}

def f(params):
    X_ = X[:]

    if 'normalize' in params:
        if params['normalize'] == 1:
            normalize = MinMaxScaler()
            X_ = normalize.fit_transform(X_)
    del params['normalize']

    if 'scale' in params:
        if params['scale'] == 1:
            scale = StandardScaler()
            X_ = scale.fit_transform(X_)
    del params['scale']

    clf = SVC(**params)
    acc = cross_val_score(clf, X_, y).mean()

    return {'loss': -acc, 'status': STATUS_OK}


trials = Trials()
best = fmin(f
            , space4svm
            , algo=tpe.suggest
            , max_evals=100
            , trials=trials
           )
print('best:\n',best)

parameters = ['C', 'kernel', 'gamma', 'scale', 'normalize']
cols = len(parameters)
f, axes = plt.subplots(nrows=1, ncols=cols, figsize=(20,5))
cmap = plt.cm.jet
```

```
for i, val in enumerate(parameters):
    xs = np.array([t['misc']['vals'][val] for t in trials.trials]).ravel()
    ys = [-t['result']['loss'] for t in trials.trials]
    xs, ys = zip(*sorted(zip(xs, ys)))
    axes[i].scatter(xs
                    , ys
                    , s=20
                    , linewidth=0.01
                    , alpha=0.25
                    , c=cmap(float(i)/len(parameters))
                   )
    axes[i].set_title(val)
    axes[i].set_ylim([0.9, 1.0])

"""输出如下"""
 {'C': 0.7168209011868267
  , 'gamma': 3.7409805767298074
  , 'kernel': 0
  , 'normalize': 0
  , 'scale': 0}
```

Here is what we get:



Again, scaling and normalizing do not help. The first choice of kernel funcion is the best (`linear`), the best `C` value is `0.7168209011868267` , and the best `gamma` is `3.7409805767298074` . This set of parameters results in 99.3 % classification accuracy.

3.1.10 Decision Trees

- We will only attempt to optimize on a few parameters of decision trees. Here is the code.

  我们只会尝试优化决策树的几个参数。

```
from sklearn import datasets
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
```

```python
import numpy as np

iris = datasets.load_iris()
X_original = iris.data
y_original = iris.target

space4dt = {
    'max_depth': hp.choice('max_depth', range(1,20)),
    'max_features': hp.choice('max_features', range(1,5)),
    'criterion': hp.choice('criterion', ["gini", "entropy"]),
    'scale': hp.choice('scale', [0, 1]),
    'normalize': hp.choice('normalize', [0, 1])
}

def f(params):
    X_ = X[:]

    if 'normalize' in params:
        if params['normalize'] == 1:
            normalize = MinMaxScaler()
            X_ = normalize.fit_transform(X_)
    del params['normalize']

    if 'scale' in params:
        if params['scale'] == 1:
            scale = StandardScaler()
            X_ = scale.fit_transform(X_)
    del params['scale']

    clf = DecisionTreeClassifier(**params)
    acc = cross_val_score(clf, X_, y).mean()

    return {'loss': -acc, 'status': STATUS_OK}

trials = Trials()
best = fmin(f
            , space4dt
            , algo=tpe.suggest
            , max_evals=300
            , trials=trials)
print('best:',best)
```

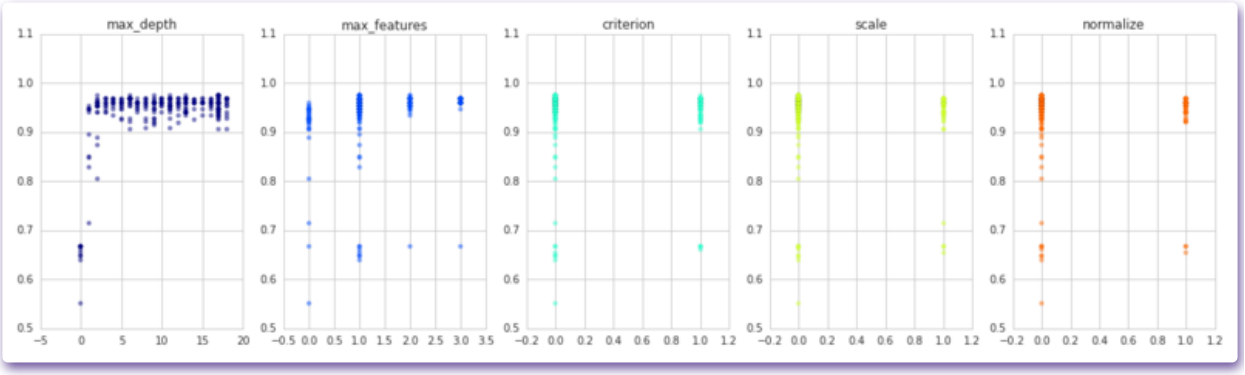
The output is the following, which gives 97.3 % accuracy.

```
"""输出如下"""
best: {'criterion': 0
       , 'max_depth': 6
       , 'max_features': 2
       , 'normalize': 0
       , 'scale': 0}
```

Here are the plots. We can see that there is little difference in performance with different values of `scale`, `normalize`, and `criterion`.

我们可以看到，在 `scale`、`normalize` 和 `criterion` 的不同值下，性能差异不大。

```
parameters = ['max_depth', 'max_features'
              , 'criterion', 'scale', 'normalize'] # decision tree
cols = len(parameters)
f, axes = plt.subplots(nrows=1, ncols=cols, figsize=(20,5))
cmap = plt.cm.jet
for i, val in enumerate(parameters):
    xs = np.array([t['misc']['vals'][val] for t in trials.trials]).ravel()
    ys = [-t['result']['loss'] for t in trials.trials]
    xs, ys = zip(*sorted(zip(xs, ys)))
    ys = np.array(ys)
    axes[i].scatter(xs
                    , ys
                    , s=20
                    , linewidth=0.01
                    , alpha=0.5
                    , c=cmap(float(i)/len(parameters))
                   )
    axes[i].set_title(val)
    #axes[i].set_ylim([0.9,1.0])
```



3.1.11 Random Forests

- Let's see what's happending with an ensemble classifier, Random Forest, which is just a collection of decision trees trained on different even-sized partitions of the data, each of which votes on an output class, with the majority class being chosen as the prediction.

  让我们来看看集成分类器随机森林发生了什么，随机森林只是在不同分区数据上训练的决

策树集合，每个分区都对输出类进行投票，并将绝大多数类的选择为预测

```
from sklearn import datasets
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
import numpy as np

iris = datasets.load_iris()
X = iris.data
y = iris.target

space4rf = {
    'max_depth': hp.choice('max_depth', range(1,20)),
    'max_features': hp.choice('max_features', range(1,5)),
    'n_estimators': hp.choice('n_estimators', range(1,20)),
    'criterion': hp.choice('criterion', ["gini", "entropy"]),
    'scale': hp.choice('scale', [0, 1]),
    'normalize': hp.choice('normalize', [0, 1])
}

best = 0

def f(params):
    global best
    X_ = X[:]

    if 'normalize' in params:
        if params['normalize'] == 1:
            normalize = MinMaxScaler()
            X_ = normalize.fit_transform(X_)
    del params['normalize']

    if 'scale' in params:
        if params['scale'] == 1:
            scale = StandardScaler()
            X_ = scale.fit_transform(X_)
    del params['scale']

    clf = RandomForestClassifier(**params)
    acc = cross_val_score(clf, X_, y).mean()

    if acc > best:
        best = acc
        print('new best:', best, params)
    return {'loss': -acc, 'status': STATUS_OK}

trials = Trials()
```

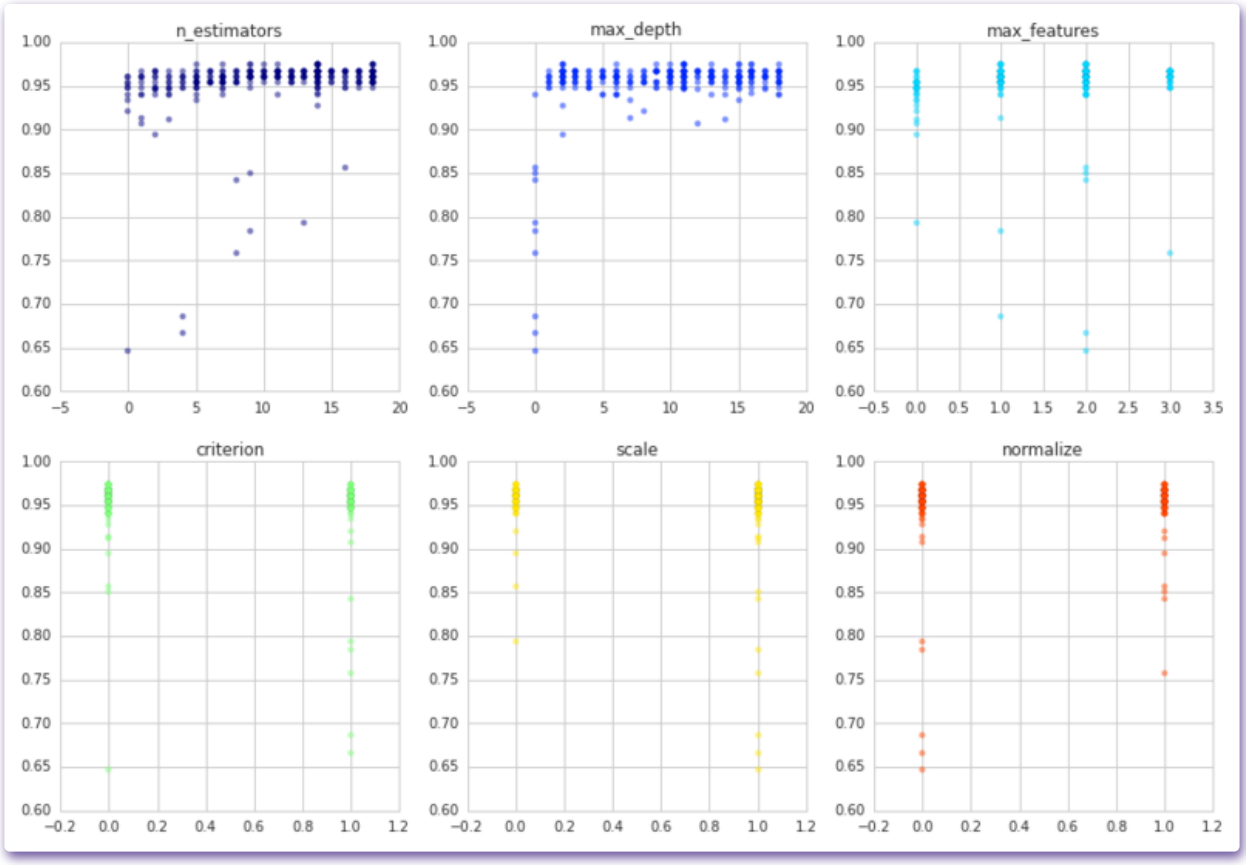
```
best = fmin(f, space4rf, algo=tpe.suggest, max_evals=300, trials=trials)
print('best:\n',best)


"""输出如下"""
new best:

0.9666666666666668

{'criterion': 'gini'
 , 'max_depth': 8
 , 'max_features': 4
 , 'n_estimators': 9
}

new best:

0.9800000000000001

{'criterion': 'gini'
 , 'max_depth': 18
 , 'max_features': 3
 , 'n_estimators': 6
}

best:
 {'criterion': 0
  , 'max_depth': 17
  , 'max_features': 2
  , 'n_estimators': 5
  , 'normalize': 0
  , 'scale': 0
 }
```

Again, we only get 97.3 % accuracy, same as decision tree.

Here is the code to plot the parameters:

```
parameters = ['n_estimators', 'max_depth'
              , 'max_features', 'criterion'
              , 'scale', 'normalize']
f, axes = plt.subplots(nrows=2, ncols=3, figsize=(15,10))
cmap = plt.cm.jet
for i, val in enumerate(parameters):
    print(i, val)
    xs = np.array([t['misc']['vals'][val] for t in trials.trials]).ravel()
    ys = [-t['result']['loss'] for t in trials.trials]
    xs, ys = zip(*sorted(zip(xs, ys)))
    ys = np.array(ys)
    axes[i//3,i%3].scatter(xs
```

```
                                , ys
                                , s=20
                                , linewidth=0.01
                                , alpha=0.5
                                , c=cmap(float(i)/len(parameters))
                               )
        axes[i//3,i%3].set_title(val)
        #axes[i/3,i%3].set_ylim([0.9,1.0])

    """输出如下"""
    0 n_estimators
    1 max_depth
    2 max_features
    3 criterion
    4 scale
    5 normalize
```



3.1.12 All Together Now

- While it is fun and instructive to automatically tune the parameters of one model - SVM or KNN, for example - it is more useful to tune them all at once and arrive at a best model overall. This allows us to compare all parameters and all models at once, which gives us the best model. Here is the code.

  自动调整一个模型的参数（如SVM或KNN）非常有趣并且具有启发性，但同时调整它们并取得全局最佳模型则更有用。这使我们能够一次比较所有参数和所有模型，因此为我们提

供了最佳模型。代码如下：

```
from sklearn import datasets
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import BernoulliNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
import numpy as np

digits = datasets.load_digits()
X = digits.data
y = digits.target
print(X.shape, y.shape)

def hyperopt_train_test(params):
    t = params['type']
    del params['type']
    if t == 'naive_bayes':
        clf = BernoulliNB(**params)
    elif t == 'svm':
        clf = SVC(**params)
    elif t == 'dtree':
        clf = DecisionTreeClassifier(**params)
    elif t == 'knn':
        clf = KNeighborsClassifier(**params)
    else:
        return 0
    return cross_val_score(clf, X, y).mean()

space = hp.choice('classifier_type', [
    {
        'type': 'naive_bayes',
        'alpha': hp.uniform('alpha', 0.0, 2.0)
    },
    {
        'type': 'svm',
        'C': hp.uniform('C', 0, 10.0),
        'kernel': hp.choice('kernel', ['linear', 'rbf']),
        'gamma': hp.uniform('gamma', 0, 20.0)
    },
    {
        'type': 'randomforest',
        'max_depth': hp.choice('max_depth', range(1,20)),
        'max_features': hp.choice('max_features', range(1,5)),
        'n_estimators': hp.choice('n_estimators', range(1,20)),
        'criterion': hp.choice('criterion', ["gini", "entropy"]),
        'scale': hp.choice('scale', [0, 1]),
        'normalize': hp.choice('normalize', [0, 1])
```

```
        },
        {
            'type': 'knn',
            'n_neighbors': hp.choice('knn_n_neighbors', range(1,50))
        }
    ])

    count = 0
    best = 0
    def f(params):
        global best, count
        count += 1
        acc = hyperopt_train_test(params.copy())
        if acc > best:
            print('new best:', acc, 'using', params['type'])
            best = acc
        if count % 50 == 0:
            print('iters:', count, ', acc:', acc, 'using', params)
        return {'loss': -acc, 'status': STATUS_OK}

    trials = Trials()
    best = fmin(f
                , space
                , algo=tpe.suggest
                , max_evals=1500
                , trials=trials
               )
    print('best:\n','best:')
```

This code takes a while to run since we increased the number of evaluations: `max_evals=1500`. There is also added output to update you when a new `best` accuracy is found. Curious as to why using this method does not find the best model that we found above: `SVM` with `kernel=linear`, `C=1.416`, and `gamma=15.042`.

由于我们增加了评估数量，此代码需要一段时间才能运行完：`max_evals=1500`。当找到新的最佳准确率时，它还会添加到输出用于更新。好奇为什么使用这种方法没有找到前面的最佳模型：参数为 `kernel=linear`，`C=1.416`，`gamma=15.042` 的 `SVM`。

3.1.13 Conclusion

- We have covered simple examples, like minimizing a deterministic linear function, and complicated examples, like tuning random forest parameters. The documentation for `hyperopt` is **here**. Another good blog on hyperopt is **this one** by FastML. A SciPy Conference paper by the `hyperopt` authors is **Hyperopt: A Python Library for Optimizing the Hyperparameters of Machine Learning Algorithms**, with an accompanying **video tutorial**. A more techical treatment of the engineering ins and outs is **Making a Science of Model Search**.

  我们已经介绍了简单的例子，如最小化确定的线性函数，以及复杂的例子，如调整随机森林参数。`hyperopt` 的官方文档在 **这里**。另一篇有关 hyperopt 不错的博客位于 FastML 站点

（译注：已无效）。`hyperopt` 作者的SciPy会议论文是 **Hyperopt: A Python Library for Optimizing the Hyperparameters of Machine Learning Algorithms**，伴随一起的 **视频教程**。关于工程来龙去脉的更多技术处理细节可以参见 **Making a Science of Model Search**。

The techniques in this post can be used in many domains other than machine learning, such as tuning the `epsilon` parameter in an epsilon-greedy multi-armed bandit, or the parameters passed to a graph generator to make a synthetic network that has certain properties. We will write more on that later.

这篇文章中的技术可以用于机器学习以外的许多领域，例如调整 epsilon-greedy 多臂老虎机中的“epsilon”参数，或者将参数传递给图形生成器以制作具有一定 特性。 我们稍后会写更多。

3.2 lightgbm 分类例子

- 以二分类，sklearn中的breast数据集为例子

```
from tqdm import notebook
import pandas as pd
from sklearn.datasets import load_breast_cancer
from lightgbm import LGBMClassifier as LGB
from sklearn.model_selection import train_test_split
from hyperopt import hp,tpe,STATUS_OK,partial,fmin,Trials
from sklearn.metrics import roc_auc_score as AUC
import numpy as np

cancer = load_breast_cancer()
df = pd.DataFrame(cancer.data,columns=['_'.join(i.split(' ')) for i in cancer.feature_names])
y  = pd.Series(cancer.target)

X_train,X_test,y_train,y_test = train_test_split(df,y,test_size=0.3)

training_features=list(df.columns)

best_auc = 0

def lgb_factory(params):

    global  best_auc
    lgb_bayes = LGB(**params)
    lgb_bayes.fit(X_train[training_features],y_train)
    y_prob_bayes = lgb_bayes.predict_proba(X_test[training_features])[:,1]

    auc = AUC(y_test,y_prob_bayes)
    print(auc)

    if auc > best_auc:
```

```
        best_auc =auc

    return -auc

lgb_space = {"max_depth":hp.randint("max_depth",30)
,"n_estimators":hp.choice("n_estimators",np.arange(10,1001,1).tolist())
            ,"learning_rate":hp.uniform('learning_rate',0.001,0.5)
,"min_child_samples":hp.choice("min_child_samples",np.arange(1,21,1).tolist())
,"num_leaves":hp.choice("num_leaves",np.arange(2,100,1).tolist())
,"min_child_weight":hp.uniform('min_child_weight',0.001,1)
,"colsample_bytree":hp.uniform('colsample_bytree',0.8,1)
            ,"reg_alpha":hp.uniform("reg_alpha",0.00,5)
            ,"reg_lambda":hp.uniform("reg_lambda",0.00,5)
,"random_state":hp.choice("random_state",np.arange(1,1000,1).tolist())
,"scale_pos_weight":hp.uniform("scale_pos_weight",0,10)
            }

trials = Trials()
algo=partial(tpe.suggest,n_startup_jobs=8)
best = fmin(fn=lgb_factory
           ,space=lgb_space # 参数域
           ,algo=algo
           ,max_evals=10 # 迭代次数
           ,trials=trials
           )



"""输出"""
0.9981439177612792
0.9964306110793832

0.9984294688749287

0.9985722444317533

0.99785836664763

0.9970017133066819

0.9961450599657339
```

```
0.9984294688749287

0.9984294688749287

0.5

100%|██████████| 10/10 [00:04<00:00,  2.19trial/s, best loss: -0.9985722444317533]

```